

数据结构与算法 B

common-21 春-期末

一、选择题 (30 分, 每小题 2 分)

1. 下列不影响算法时间复杂性的因素有 ()。

- | | |
|-----------|-----------|
| (A) 问题的规模 | (C) 计算结果 |
| (B) 输入值 | (D) 算法的策略 |

解答:

选 C。算法时间复杂度由问题规模、输入数据和算法策略决定, 与计算结果无关。

2. 链表不具有的特点是 ()。

- | | |
|------------------|-------------------|
| (A) 可随机访问任意元素 | (C) 不必事先估计存储空间 |
| (B) 插入和删除不需要移动元素 | (D) 所需空间与线性表长度成正比 |

解答:

选 A。链表是顺序访问结构, 不支持随机访问。

3. 设有三个元素 X, Y, Z 顺序进栈 (进的过程中允许出栈), 下列得不到的出栈排列是 ()。

- | | |
|-----------|-----------|
| (A) XYZ | (C) ZXY |
| (B) YZX | (D) ZYX |

解答:

选 C。 ZXY 不可能: 若 Z 第一个出栈, 则 X, Y 已在栈中且 Y 在栈顶, 之后必须先出 Y 再出 X , 不能得到 ZXY 。

4. 判定一个队列 Q (申请数组空间为 m) 为空的条件是 ()。

- | |
|----------------------------------|
| (A) $Q.rear == Q.front$ |
| (B) $Q.front != Q.rear$ |
| (C) $Q.front == (Q.rear+1) \% m$ |
| (D) $Q.front != (Q.rear+1) \% m$ |

解答:

选 A。循环队列判空条件为 $rear == front$, 判满条件为 $(rear+1) \% m == front$ 。

5. 若定义二叉树中根结点的层数为 0, 树的高度等于其结点的最大层数加一。则当某二叉树的前序遍历序列和后序遍历序列正好相反, 则该二叉树一定是 ()。

- | | |
|--------------|--------------|
| (A) 空或只有一个结点 | (C) 任一结点无左孩子 |
| (B) 高度等于其结点数 | (D) 任一结点无右孩子 |

解答:

选 B。先序和后序相反意味着每层只有一个结点, 树退化为单链, 高度等于结点数。

6. 任意一棵二叉树中, 所有叶结点在前序遍历序列、中序遍历序列和后序遍历序列中的相对次序 ()。

- (A) 发生改变 (C) 不能确定
(B) 不发生改变 (D) 以上都不对

解答：

选 B。叶子结点在三种遍历中均保持从左到右的相对顺序。

7. 假设线性表中每个元素有两个数据项 key1 和 key2，现对线性表按以下规则进行排序：先根据 key1 的值进行非递减排序；在 key1 值相同的情况下，再根据 key2 的值进行非递减排序。满足这种要求的排序方法是 ()。

- (A) 先按 key1 冒泡排序，再按 key2 直接选择排序
(B) 先按 key2 冒泡排序，再按 key1 直接选择排序
(C) 先按 key1 直接选择排序，再按 key2 冒泡排序
(D) 先按 key2 直接选择排序，再按 key1 冒泡排序

解答：

选 D。两关键字排序通常应先按次要关键字 key2 排序，再按主要关键字 key1 进行稳定排序。冒泡排序稳定，能在第二趟按 key1 排序时保留 key1 相同记录之间的 key2 次序；直接选择排序通常不稳定，因此不能放在第二趟作为主关键字排序。

8. 有 n 个整数，找到其中最小整数需要比较次数至少是 () 次。

- (A) n (C) $n^2 - 1$
(B) $\log_2 n$ (D) $n - 1$

解答：

选 D。找最小值至少需要 $n - 1$ 次比较 (每个元素至少参与一次比较，锦标赛形式)。

9. n 个顶点的无向完全图的边数为 ()。

- (A) $n(n - 1)$ (C) $n(n - 1)/2$
(B) $n(n + 1)$ (D) $n(n + 1)/2$

解答：

选 C。无向完全图每对顶点间有一条边，共 $C_n^2 = n(n - 1)/2$ 条。

10. 设无向图 $G = (V, E)$ ， $G' = (V', E')$ ，如果 G' 是 G 的生成树，则下面说法中错误的是 ()。

- (A) G' 是 G 的子图 (C) G' 是 G 的极小连通子图且 $V = V'$
(B) G' 是 G 的连通分量 (D) G' 是 G 的一个无环子图

解答：

选 B。生成树是包含所有顶点的极小连通子图，但不一定是连通分量 (连通分量是极大连通子图)。

11. 有一个散列表，其散列函数为 $h(\text{key}) = \text{key} \bmod 13$ ，使用再散列函数 $h_2(\text{key}) = \text{key} \bmod 3$ 解决碰撞。当前散列表状态如下，其中空白表示该地址尚未存放关键词：

地址	0	1	2	3	4	5	6	7	8	9	10	11	12
关键词		38											25

从表中检索关键词 38 需进行几次关键词比较 ()。

- (A) 1
- (B) 2
- (C) 3
- (D) 4

解答:

选 B。 $h(38) = 38 \bmod 13 = 12$ ，第一次比较地址 12 中的 25，不匹配；步长为 $h_2(38) = 2$ ，下一探查地址为 $(12 + 2) \bmod 13 = 1$ ，第二次比较找到 38。

12. 在一棵度为 3 的树中，度为 3 的结点个数为 2，度为 2 的结点个数为 1，则度为 0 的结点个数为 ()。

- (A) 4
- (B) 5
- (C) 6
- (D) 7

解答:

选 C。树中总边数 = 总度数 = $3 \times 2 + 2 \times 1 + n_1$ ，总结点数 = $2 + 1 + n_1 + n_0$ ，由边数 = 结点数 - 1 得 $n_0 = 6$ 。

13. 由同一组关键字集合构造的各棵二叉排序树 ()。

- (A) 其形态不一定相同，但平均查找长度相同
- (B) 其形态不一定相同，平均查找长度也不一定相同
- (C) 其形态均相同，但平均查找长度不一定相同
- (D) 其形态均相同，平均查找长度也都相同

解答:

选 B。BST 形态取决于插入顺序，不同形态影响查找效率。

14. 允许表达式内多种括号混合嵌套，检查表达式中括号是否正确配对的算法，通常选用 ()。

- (A) 栈
- (B) 线性表
- (C) 队列
- (D) 二叉排序树

解答:

选 A。栈的后进先出特性适合括号匹配检查。

15. 下列选项中最适合实现数据的频繁增删及高效查找的组织结构是 ()。

- (A) 有序表
- (B) 堆排序
- (C) 二叉排序树
- (D) 快速排序

解答:

选 C。BST 支持 $O(\log n)$ 的查找与 $O(\log n)$ 的插入/删除 (平均情况)。

二、判断题 (10 分，每小题 1 分；对 Y，错 N)

16. 考虑一个长度为 n 的顺序表中各个位置插入新元素的概率相同，则顺序表的插入算法平均时间复杂度为 $O(n)$ 。()

解答:

答案: Y。顺序表插入时，需要把插入位置及其后的元素整体后移。若插入到第 i 个位置后面，则移

动元素个数与 i 有关；各位置等概率时，平均移动次数约为 $n/2$ ，因此平均时间复杂度为 $O(n)$ 。

17. 希尔排序算法的每一趟都要调用一次或多次直接插入排序算法，所以其效率比直接插入排序算法差。()

解答：

答案：N。希尔排序虽然在每个增量分组内使用直接插入排序思想，但它先让相隔较远的元素交换，使序列逐步趋于有序；后续增量变小时，插入排序的代价会明显降低。因此不能因为“调用插入排序”就判断它一定更差。

18. 直接插入排序、冒泡排序、Shell 排序都是在数据正序的情况下比数据在逆序的情况下要快。()

解答：

答案：Y。直接插入排序在正序时几乎不移动元素；带提前结束标志的冒泡排序在正序时一趟即可结束；Shell 排序在序列较有序时分组插入的移动量也更小。逆序通常导致更多比较和移动。

19. 由于碰撞的发生，基于散列表的检索仍然需要进行关键码对比，并且关键码的比较次数仅取决于选择的散列函数与处理碰撞的方法两个因素。()

解答：

答案：N。散列表检索次数不仅与散列函数和冲突处理方法有关，还与装填因子、关键字分布、已有冲突链或探查序列长度有关。装填因子越高，平均冲突和比较次数往往越多。

20. 若有一个叶子结点是二叉树中某个子树的前序遍历结果序列的最后一个结点，则它一定是该子树的中序遍历结果序列的最后一个结点。()

解答：

答案：N。反例：某子树根为 A ，只有左孩子 B 。其前序遍历为 A, B ，最后一个结点 B 是叶子；但中序遍历为 B, A ，最后一个结点是根 A 。所以该命题不成立。

21. 若某非空二叉树的前序遍历序列和后序遍历序列正好相同，则该二叉树只有一个根结点。()

解答：

答案：Y。前序遍历第一个结点一定是根，后序遍历最后一个结点一定是根。若两序列完全相同，则第一个结点也必须等于最后一个结点；非空且结点互异时，只能说明序列长度为 1，即二叉树只有根结点。

22. 有 n 个结点的二叉排序树有多种；若按最坏情况下的查找比较次数衡量，树高最小的二叉排序树搜索效率最好。()

解答：

答案：Y。二叉排序树查找时，每向下一层就多比较一次，最坏比较次数与树高成正相关。树高越小，最坏情况下从根到叶的路径越短，因此按最坏情况衡量搜索效率更好。

23. 强连通分量是有向图中的最小强连通子图。()

解答：

答案：N。强连通分量是有向图中的极大强连通子图，即再加入任何相邻顶点都会破坏强连通性；它不是“最小”强连通子图。单个顶点也可看作强连通子图，但通常不是强连通分量。

24. 用邻接矩阵法存储一个图时，在不考虑压缩存储的情况下，所占用的存储空间大小只与图中结点个数有关，而与图的边数无关。()

解答：

答案：Y。邻接矩阵为每一对顶点预留一个表项，若图有 n 个顶点，则矩阵大小为 $n \times n$ 。无论边多还是边少，都要存储这个矩阵，因此空间复杂度为 $O(n^2)$ ，与边数无关。

25. 若定义一个有向图的根是指可以从这个结点到达图中任意其他结点，则可知一个有向图中至少有一个根。()

解答：

答案：N。有向图不一定存在能到达所有其他顶点的结点。例如两个互不连通的强连通分量之间没有路径时，任何一个顶点都无法到达另一分量中的顶点，因此不存在这样的根。

三、填空题 (20 分，每题 2 分)

26. 线性表的顺序存储与链式存储是两种常见存储形式；当表元素有序排序进行二分检索时，应采用 _____ 存储形式。

解答：

答案：顺序存储。二分查找每一步都要直接访问中间位置元素，顺序表支持按下标随机访问，时间为 $O(1)$ ；链表只能从头顺序扫描到中间结点，无法高效二分。

27. 设环形队列的容量为 20(单元编号 0 到 19)，经过一系列的入队和出队运算后，队头变量 $front=18$ ，队尾变量 $rear=11$ ，则环形队列中有 _____ 个元素。

解答：

答案：13。循环队列中元素个数通常计算为 $(rear - front + m) \bmod m$ 。代入 $m = 20$ 得 $(11 - 18 + 20) \bmod 20 = 13$ 。

28. 一棵含有 101 个结点的二叉树中有 36 个叶子结点，度为 2 的结点个数是 _____，度为 1 的结点个数是 _____。

解答：

答案：35，30。二叉树中叶子数 n_0 与度为 2 的结点数 n_2 满足 $n_0 = n_2 + 1$ ，故 $n_2 = 36 - 1 = 35$ 。总结点数 $n = n_0 + n_1 + n_2$ ，所以 $n_1 = 101 - 36 - 35 = 30$ 。

29. 已知二叉树的前序遍历结果为 ADC，这棵二叉树的树型有 _____ 种可能。

解答：

答案：5。只有前序序列且含 3 个互异结点时，不能唯一确定二叉树形态。含 3 个结点的二叉树形态数为 Catalan 数 $C_3 = \frac{1}{4} \binom{6}{3} = 5$ 。

30. 已知二叉树的中序序列为 DGBAECF，后序序列为 GDBEFCA，该二叉树的先序序列是 _____。

解答：

答案：ABDGCEF。后序最后一个结点 A 为根；中序按 A 分为左子树 DGB 和右子树 ECF。左子树后序为 GDB，根为 B，其左子树为 DG；继续递归得到左子树先序 B, D, G。右子树后序为 EFC，根为 C，其左子树为 E、右子树为 F。合并得到先序 A, B, D, G, C, E, F。

31. 对于具有 57 个结点的完全二叉树，按层次自顶向下、同一层自左向右、从 0 开始编号。编号为 18 的结点的父结点编号为 _____，编号为 19 的结点的右子女结点编号为 _____。

解答：

答案：8, 40。从 0 开始编号的完全二叉树中，结点 i 的父结点编号为 $\lfloor (i-1)/2 \rfloor$ ，右孩子编号为 $2i+2$ 。故 18 的父结点为 $\lfloor 17/2 \rfloor = 8$ ，19 的右孩子为 $2 \times 19 + 2 = 40$ ；由于 $40 \leq 56$ ，该孩子存在。

32. 有 n 个数据对象的二路归并排序中，每趟归并的时间复杂度为 _____。

解答：

答案： $O(n)$ 。二路归并排序的一趟归并会把当前各有序子序列两两合并，所有元素在这一趟中总共被扫描和移动常数次，因此每趟代价与元素总数成线性关系。

33. 对一组记录进行降序排序，其关键码为 (46, 70, 56, 38, 40, 80, 60, 22)，采用初始步长为 4 的希尔排序，第一趟扫描的结果是 _____，而采用归并排序第一轮归并的结果是 _____。

解答：

答案：(46, 80, 60, 38, 40, 70, 56, 22)；(70, 46, 56, 38, 80, 40, 60, 22)。希尔排序第一趟按步长 4 分组，比较 (46, 40)、(70, 80)、(56, 60)、(38, 22) 并按降序整理，得到第一组结果。归并排序第一轮把相邻单元素两两按降序归并，所以 (46, 70) 变为 (70, 46)，(56, 38) 不变，(40, 80) 变为 (80, 40)，(60, 22) 不变。

34. 如果只想得到 1000 个元素的序列中最小的前 5 个元素，在冒泡排序、快速排序、堆排序和归并排序中，哪种算法最快？ _____

解答：

答案：堆排序。只需要前 5 个最小元素时，不必完全排序。可建立小根堆后连续取出 5 次堆顶，代价约为 $O(n + 5 \log n)$ ，比完整归并排序、快速排序或冒泡排序更适合该需求。

35. 如果一个图结点多而边少 (稀疏图)，适宜采用 _____ 方式进行存储。

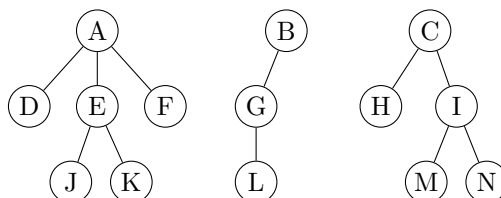
解答：

答案：邻接表。邻接矩阵固定占用 $O(n^2)$ 空间，而邻接表只保存顶点表和实际存在的边，空间为 $O(n + e)$ 。稀疏图中 e 远小于 n^2 ，因此邻接表更节省空间。

四、简答题 (24 分，每小题 6 分)

36. (森林周游) (6 分)

树周游算法可以很好地应用到森林的周游上。查看下列森林结构，请给出其深度优先周游序列和广度优先周游序列。



解答：

深度优先周游按先根次序依次访问各树：第一棵树为 A, D, E, J, K, F，第二棵树为 B, G, L，第三棵树为 C, H, I, M, N，因此序列为

A, D, E, J, K, F, B, G, L, C, H, I, M, N.

广度优先周游先访问各棵树的根，再逐层从左到右访问，序列为

$A, B, C, D, E, F, G, H, I, J, K, L, M, N.$

37. (Huffman 树) (6 分)

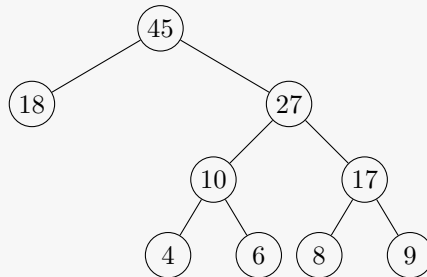
设给定五个字符，其相应的权值分别为 $\{4, 8, 6, 9, 18\}$ ，试画出相应的哈夫曼树，并计算它的带权外部路径长度 WPL。

解答：

Huffman 树构建：每次合并权值最小的两个结点。

- 合并 4, 6 得 10
- 合并 8, 9 得 17
- 合并 10, 17 得 27
- 合并 18, 27 得 45

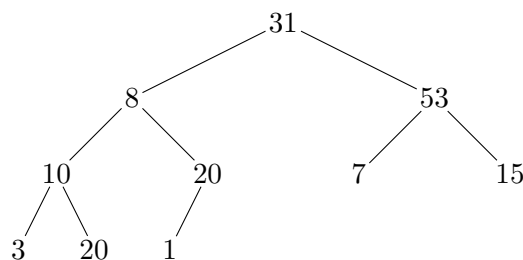
对应 Huffman 树可画为：



WPL 等于各次合并权值之和： $10 + 17 + 27 + 45 = 99$ 。等价地，4, 6, 8, 9 的深度为 3，18 的深度为 1，故 $WPL = 4 \times 3 + 6 \times 3 + 8 \times 3 + 9 \times 3 + 18 \times 1 = 99$ 。

38. (堆操作) (6 分)

下图是一棵完全二叉树：

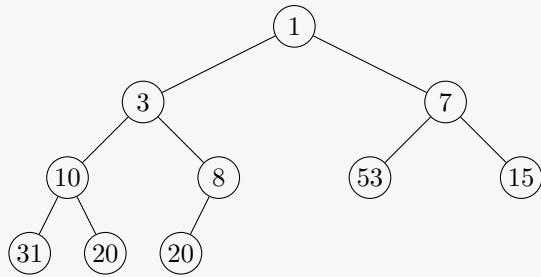


- 1) 请根据初始建堆算法对该完全二叉树建堆，请画出构建的小根堆；(2 分)
- 2) 基于 (1) 中得到的堆，删除其中的最小元素，请用图给出堆的调整过程；(2 分)
- 3) 基于 (1) 中得到的堆，向其中插入元素 2，请给出堆的调整过程。(2 分)

解答：

- 1) 原树按层次序列为 (31, 8, 53, 10, 20, 7, 15, 3, 20, 1)。自最后一个非叶结点向前下滤，可得小根堆层次序列

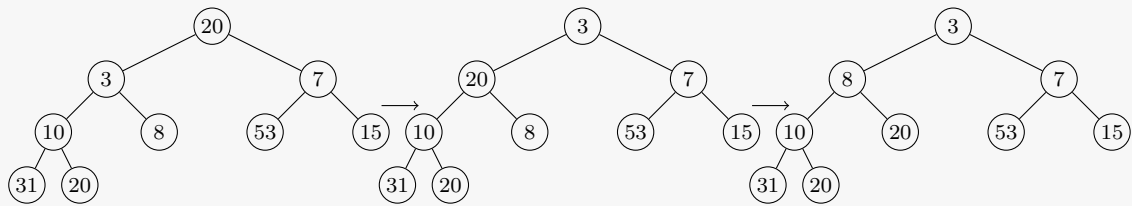
(1, 3, 7, 10, 8, 53, 15, 31, 20, 20)。



2) 删除最小元素 1 后，以末尾元素 20 补到根并下滤：

$(20, 3, 7, 10, 8, 53, 15, 31, 20) \rightarrow (3, 8, 7, 10, 20, 53, 15, 31, 20).$

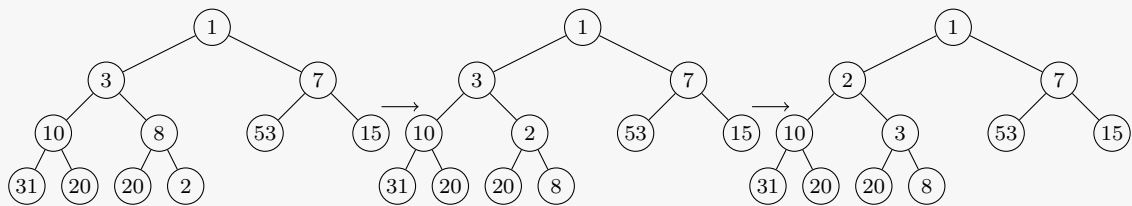
调整过程如下：



3) 插入 2 后先放在末尾，再依次与父结点 8,3 交换，结果为

$(1, 2, 7, 10, 3, 53, 15, 31, 20, 20, 8).$

调整过程如下：



39. (Prim 算法求最小生成树) (6 分)

已知图 G 的顶点集合 $V = \{V_0, V_1, V_2, V_3, V_4\}$ ，邻接矩阵如下 (∞ 表示无边)：

$$\begin{pmatrix} 0 & 7 & \infty & 4 & 2 \\ 7 & 0 & 9 & 1 & 5 \\ \infty & 9 & 0 & 3 & \infty \\ 4 & 1 & 3 & 0 & 10 \\ 2 & 5 & \infty & 10 & 0 \end{pmatrix}$$

- 1) 根据 Prim 算法，求图 G 从顶点 V_0 出发的最小生成树；(2 分)
- 2) 给出每一步执行后的 mst 数组。(4 分)

解答：
从 V_0 出发，逐步选择跨越当前顶点集的最小边。执行状态如下，其中 U 表示已加入生成树的顶点集， $lowcost$ 为到 U 的最小边权。

步骤	新加入顶点	U	当前最小边
初始	V_0	$\{V_0\}$	-
1	V_4	$\{V_0, V_4\}$	$(V_0, V_4), 2$
2	V_3	$\{V_0, V_4, V_3\}$	$(V_0, V_3), 4$
3	V_1	$\{V_0, V_4, V_3, V_1\}$	$(V_3, V_1), 1$
4	V_2	$\{V_0, V_4, V_3, V_1, V_2\}$	$(V_3, V_2), 3$

因此最小生成树为 $\{(V_0, V_4), (V_0, V_3), (V_3, V_1), (V_3, V_2)\}$ ，总权值为 10。

五、算法题 (16 分，每小题 8 分)

40. (根据前序求后序) 下列 Python 函数的功能是：根据一个满二叉树的前序遍历序列，求其后序遍历序列。参数 s 为当前子树在前序序列中的起点， $length$ 为当前子树结点数。请补全空白 (假设序列长度不超过 32)。

```
def preorder_to_postorder(seq, s, length):
    if length == 1:
        return _____ # (1)

    sub_len = _____ # (2)
    s1 = s + 1
    s2 = _____ # (3)
    left = preorder_to_postorder(seq, s1, sub_len)
    right = _____ # (4)
    return _____ # (5)
```

解答：

填空答案：

- (1) `seq[s]`
- (2) `(length - 1) // 2`
- (3) `s + sub_len + 1`
- (4) `preorder_to_postorder(seq, s2, sub_len)`
- (5) `left + right + seq[s]`

满二叉树左右子树结点数相同，均为 $(length - 1)/2$ 。前序序列的第一个元素是根，之后依次是左子树和右子树；后序遍历顺序为“左子树、右子树、根”，因此返回左、右子树后序结果再拼接根结点。

41. (深度优先周游) 阅读下列 Python 程序，完成图的深度优先周游算法。图采用邻接表表示：
`graph["vertices"]` 存放顶点标记，`graph["adj"][i]` 存放顶点 i 的所有邻接点编号。

```
def dfs_in_list(graph, visited, i):
    print(f"node: {graph['vertices'][i]}")
    visited[i] = True
    for j in _____: # (1)
        if _____: # (2)
            _____ # (3)

def traverse_dfs(graph):
    n = len(graph["vertices"])
    visited = [False] * n
    for i in range(n):
        if _____: # (4)
            dfs_in_list(graph, visited, i)
```

解答：

填空答案：

- (1) `graph["adj"][i]`
- (2) `not visited[j]`
- (3) `dfs_in_list(graph, visited, j)`
- (4) `not visited[i]`

DFS 的核心是：访问当前顶点后，依次扫描当前顶点邻接表中的顶点；对尚未访问的邻接点递归。外层循环保证即使图不连通，也能从每个未访问顶点启动一次 DFS，从而完成全图周游。

数据结构与算法 B

common-22 春-期末

一、选择题 (30 分, 每小题 2 分)

1. 双向链表中的每个结点有两个引用域 `prev` 和 `next`, 分别引用当前结点的前驱与后继。设 `p` 引用链表中的一个结点, `q` 引用一待插入结点, 现要求在 `p` 前插入 `q`, 正确的插入操作为 ()。

- (A) `p.prev=q; q.next=p; p.prev.next=q; q.prev=p.prev;`
- (B) `q.prev=p.prev; p.prev.next=q; q.next=p; p.prev=q.next;`
- (C) `q.next=p; p.next=q; p.prev.next=q; q.next=p;`
- (D) `p.prev.next=q; q.next=p; q.prev=p.prev; p.prev=q;`

解答:

选 D。该操作先令原前驱指向 `q`, 再令 `q` 指向 `p`, 最后修正 `q.prev` 与 `p.prev`。

2. 给定一个 N 个相异元素构成的有序数列, 设计递归算法实现二分查找。考察递归过程中栈的使用情况, 该递归调用栈的最小容量应为 ()。

- (A) N
- (B) $N/2$
- (C) $\lfloor \log_2 N \rfloor$
- (D) $\lceil \log_2(N+1) \rceil$

解答:

选 D。递归二分查找的最大递归深度等于判定树的高度, 量级为 $\lceil \log_2(N+1) \rceil$ 。

3. 数据结构有三个基本要素: 逻辑结构、存储结构以及基于结构定义的行为 (运算)。下列概念中, () 属于存储结构。

- (A) 线性表
- (B) 链表
- (C) 字符串
- (D) 二叉树

解答:

选 B。链表是线性表的一种链式存储结构; 其余选项通常指逻辑结构。

4. 采用数组 $Q[0..m-1]$ 实现循环队列, 变量 `rear` 表示队尾元素的实际位置, 添加结点时按 `rear = (rear + 1) % m` 移动, 变量 `length` 表示当前队列中的元素个数, 则队首元素的实际位置是 ()。

- (A) `rear - length`
- (B) `(1 + rear + m - length) % m`
- (C) `(rear - length + m) % m`
- (D) `m - length`

解答:

选 B。队尾为最后一个元素, 因此队首下标为 `rear - length + 1`, 取模后即选项 B。

5. 给定一棵二叉树, 若前序遍历序列与中序遍历序列相同, 则该二叉树是 ()。

- (A) 根结点无左子树的二叉树
- (B) 根结点无右子树的二叉树
- (C) 只有根结点的二叉树或非叶子结点只有左子树的二叉树
- (D) 只有根结点的二叉树或非叶子结点只有右子树的二叉树

解答：

选 D。若每个非叶结点都无左子树，则前序和中序都按“根、右子树”访问；反之若某结点有左子树，则中序会先访问其左子树，与前序不同。

6. 用 Huffman 算法构造最优二叉编码树，待编码字符权值为 $\{3, 4, 5, 6, 8, 9, 11, 12\}$ ，该树的带权外部路径长度为 ()。

- | | |
|---------|---------|
| (A) 58 | (C) 72 |
| (B) 169 | (D) 182 |

解答：

选 B。依次合并 3, 4; 5, 6; 7, 8; 9, 11; 11, 12; 15, 20; 23, 35，合并权值和为 $7+11+15+20+23+35+58 = 169$ 。

7. 若需要对存储开销约 1GB 的数据排序，而当前可用 RAM 只有 100MB，最适合的排序算法是 ()。

- | | |
|----------|----------|
| (A) 堆排序 | (C) 快速排序 |
| (B) 归并排序 | (D) 插入排序 |

解答：

选 B。外部排序常采用多路归并思想，适合数据量显著大于内存容量的情形。

8. 一个无向图 G 含有 18 条边，其中度数为 4 的顶点有 3 个，度数为 3 的顶点有 4 个，其他顶点的度数均小于 3，则 G 的顶点数至少为 ()。

- (A) 10
(B) 11
(C) 13
(D) 15

解答：

选 C。总度数为 $2|E| = 36$ 。已知顶点贡献度数 $3 \cdot 4 + 4 \cdot 3 = 24$ ，还差 12；其余顶点度数均至多为 2，至少还需 6 个顶点，所以总顶点数至少为 $3 + 4 + 6 = 13$ 。

9. 无向图 G 从 V_0 出发作深度优先遍历，访问的树边集合为

$$\{(V_0, V_1), (V_0, V_4), (V_1, V_2), (V_1, V_3), (V_4, V_5), (V_5, V_6)\}.$$

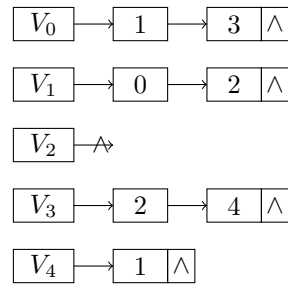
则下列哪条边不可能出现在 G 中? ()

- | | |
|------------------|------------------|
| (A) (V_0, V_2) | (C) (V_4, V_3) |
| (B) (V_4, V_6) | (D) (V_0, V_6) |

解答：

选 C。无向图 DFS 中非树边只能连接祖先与后代。 (V_4, V_3) 位于 DFS 树的两个不同分支之间，不可能作为非树边出现。

10. 已知有向图 G 的邻接边表如下图所示，



从 V_0 出发进行深度优先周游，得到的 DFS 结点序列为 ()。

- (A) V_0, V_1, V_4, V_3, V_2 (C) V_0, V_1, V_3, V_2, V_4
 (B) V_0, V_1, V_2, V_3, V_4 (D) V_0, V_2, V_1, V_3, V_4

解答：

选 B。按邻接表顺序访问： $V_0 \rightarrow V_1 \rightarrow V_2$ ，回溯后由 $V_0 \rightarrow V_3 \rightarrow V_4$ ，故序列为 V_0, V_1, V_2, V_3, V_4 。

11. 若按排序的稳定性对排序算法分类，下列算法中不稳定的是 ()。

- (A) 冒泡排序 (C) 直接插入排序
 (B) 归并排序 (D) 希尔排序

解答：

选 D。希尔排序可能使相同关键码的元素跨间隔交换，通常不稳定。

12. 下列哪组排序算法的最好情况下时间复杂度的大 O 表示相同？()

- (A) 快速排序和堆排序 (C) 快速排序和选择排序
 (B) 归并排序和插入排序 (D) 堆排序和冒泡排序

解答：

选 A。快速排序最好为 $O(n \log n)$ ，堆排序各情形均为 $O(n \log n)$ 。

13. 设结点 X 和 Y 是二叉树中任意两个结点。若在该树的先根周游序列中 X 出现在 Y 之前，而在其后根周游序列中 X 出现在 Y 之后，则 X 与 Y 的关系是 ()。

- (A) X 是 Y 的左兄弟
 (B) X 是 Y 的右兄弟
 (C) X 是 Y 的祖先
 (D) X 是 Y 的后裔

解答：

选 C。祖先结点在前序中先于后代访问，在后序中后于后代访问。

14. 森林 F 中每个结点的子结点数均不超过 2。若森林 F 中叶子结点总数为 L ，度数为 2 的结点总数为 N ，则森林 F 中树的个数为 ()。

- (A) $L - N - 1$
 (B) 无法确定
 (C) $L - N$
 (D) $N - L$

解答：

选 C。设度为 1 的结点数为 N_1 ，树的棵数为 t 。边数为 $L + N_1 + N - t$ ，也等于 $N_1 + 2N$ ，所以 $t = L - N$ 。

15. 回溯法是一类广泛使用的算法，下列叙述中不正确的是 ()。

- (A) 回溯法可以系统地搜索一个问题的所有解或者任意解
(B) 回溯法是一种既具系统性又具跳跃性的搜索算法
(C) 回溯算法需要借助队列数据结构来保存从根结点到当前扩展结点的路径
(D) 回溯法在生成解空间的任一结点时，先判断当前结点是否可能包含问题的有效解；若肯定不包含，则跳过对该结点为根的子树的搜索，逐层向祖先结点回溯

解答：

选 C。回溯通常借助递归栈或显式栈保存路径，而不是队列。

二、判断题 (10 分，每小题 1 分；对填 Y，错填 N)

16. 对任意一个连通的、无环的无向图，从图中移除任何一条边得到的图均不连通。()

解答：

答案：Y。连通且无环的无向图就是树。树中任意一条边都是桥，删除任何一条边都会把原来的树分成两个连通分量，因此所得图不再连通。

17. 给定二叉树，前序周游序列为 HGEDBFCA、中序周游序列为 EGBDHFAC 时，其后序周游序列必为 EBDGACFH。()

解答：

答案：Y。前序第一个结点 H 是根；中序序列以 H 分成左子树 EGBD 和右子树 FAC。递归划分左子树可得后序 EBDG，右子树可得后序 ACF，最后访问根 H ，合并为 EBDGACFH。

18. 对结点值在 1 到 1000 之间的二叉搜索树查找 363，以下三个序列皆可能为查找路径：

- A. 2, 252, 401, 398, 330, 344, 397, 363;
B. 925, 202, 911, 240, 912, 245, 363;
C. 935, 278, 347, 621, 299, 392, 358, 363.

()

解答：

答案：N。二叉搜索树查找路径必须满足逐步收缩的区间约束：若当前结点大于 363，下一步只能进入左子树，上界随之变小；若当前结点小于 363，下一步只能进入右子树，下界随之变大。序列 A 满足区间约束；序列 B 中，访问 911 后若目标 363 位于其左子树，则后续结点必须小于 911，但下一步出现 912，矛盾；序列 C 中，访问 347 后目标位于其右子树，后续结点必须大于 347，但之后出现 299，矛盾。因此不能说“三个序列皆可能”。

19. 构建一个含 N 个结点的 (二叉) 最小值堆，建堆时间复杂度的大 O 表示为 $O(N \log N)$ 。()

解答：

答案：N。若采用自底向上的筛选建堆方法 (常称 Floyd 建堆)，从最后一个非叶子结点开始下滤调整，整体时间复杂度为 $O(N)$ 。 $O(N \log N)$ 是逐个插入建堆的一种上界，不是通常“建堆”算法的紧确复杂度。

20. 队列是动态集合，其定义的出队列操作所移除的元素总是在集合中存在时间最长的元素。()

解答：

答案：Y。队列遵循先进先出 (FIFO) 原则，最早进入队列且尚未出队的元素会最先被删除。因此出队操作删除的是当前队列中等待时间最长的元素。

21. 任一有向图的拓扑序列既可以通过深度优先搜索求解，也可以通过宽度优先搜索求解。()

解答：

答案：Y。拓扑排序可用 DFS：按完成时间逆序输出；也可用 BFS 思想的 Kahn 算法：反复选取入度为 0 的顶点并删除其出边。两类方法都能求有向无环图的拓扑序列。

22. 对任一连通无向图 G ，其中 E 是权值最小的边，那么 E 必然属于任何一个最小生成树。()

解答：

答案：N。若权值最小的边不唯一，某一条最小权边未必出现在所有最小生成树中。例如一个三角形三条边权相同，任取两条即可构成最小生成树，第三条最小权边就不在该生成树中。若额外说明“唯一最小边”，结论才成立。

23. 对一个包含负权值边的图，Dijkstra 算法总能够给出最短路径问题的正确答案。()

解答：

答案：N。Dijkstra 算法依赖“当前最小暂定距离一旦选定就不会再被改小”的性质，该性质要求边权非负。存在负权边时，已经确定的顶点距离可能通过之后的负边再次变小，因此算法不保证正确。

24. 分治算法通常将原问题分解为几个规模较小但类似于原问题的子问题，递归求解这些子问题，然后再合并子问题的解来建立原问题的解。()

解答：

答案：Y。这正是分治法的基本框架：分解、递归求解、合并。归并排序就是典型例子，它先把序列分成较小子序列，分别排序，再把有序子序列合并。

25. 考察某个具体问题是否适合应用动态规划算法，必须判定它是否具有最优子结构性质。()

解答：

答案：Y。动态规划要求原问题的最优解能够由子问题的最优解组合而来，这称为最优子结构。若不具备该性质，就不能直接通过保存并组合子问题最优解来保证原问题最优。

三、填空题 (20 分，每题 2 分)

26. 目标串长为 n 、模式串长为 m ，朴素模式匹配算法中字符最多比较次数为 _____。

解答：

答案： $(n - m + 1)m$ ，数量级为 $O(nm)$ 。朴素匹配最多要尝试 $n - m + 1$ 个起始位置；在最坏情况下，每个起始位置都比较到模式串最后一个字符才失败或成功，因此最多比较 $(n - m + 1)m$ 次。

27. 一棵含 n 个结点的树中，只有度为 k 的分支结点和度为 0 的终端结点，则终端结点数为 _____。

解答：

答案： $\frac{(k-1)n+1}{k}$ 。设度为 k 的分支结点数为 x ，终端结点数为 n_0 ，则 $x + n_0 = n$ 。树的边数为 $n - 1$ ，

又等于所有结点度数之和 kx , 所以 $kx = n-1, x = (n-1)/k$, 从而 $n_0 = n - (n-1)/k = \frac{(k-1)n+1}{k}$ 。

28. 对关键码序列 {46, 70, 56, 38, 40, 80} 作快速排序, 以第一个记录为基准的一次划分结果为 _____。

解答:

答案: 40, 38, 46, 56, 70, 80。以 46 为基准, 划分后所有小于 46 的关键码应在其左侧, 所有大于 46 的关键码应在其右侧。按常见左右交替扫描的划分过程, 可得到 40, 38, 46, 56, 70, 80。

29. 长度为 3 的顺序表中, 查找第一个、第二个、第三个元素的概率分别为 $1/2, 1/4, 1/8$ 。若剩余概率理解为查找失败且失败需比较 3 次, 则平均比较次数为 _____。

解答:

答案: $\frac{7}{4}$ 。查找成功时, 三个位置的比较次数分别为 1, 2, 3; 成功概率总和为 $1/2 + 1/4 + 1/8 = 7/8$, 剩余失败概率为 $1/8$, 失败需比较 3 次。因此平均比较次数为 $1 \cdot \frac{1}{2} + 2 \cdot \frac{1}{4} + 3 \cdot \frac{1}{8} + 3 \cdot \frac{1}{8} = \frac{7}{4}$ 。

30. 关键码为 {19, 14, 23, 1, 68, 20, 84, 27, 55, 11, 10, 79}, 用链地址法和 $H(\text{key}) = \text{key} \bmod 13$ 构造散列表, 则地址为 1 的链中有 _____ 个记录。

解答:

答案: 4。逐个计算模 13 的余数: $14 \bmod 13 = 1, 1 \bmod 13 = 1, 27 \bmod 13 = 1, 79 \bmod 13 = 1$ 。其余关键码的余数不是 1, 所以地址 1 的链中共有 4 个记录。

31. 删除长度为 n 的顺序表第 i 个数据元素需要移动表中的 _____ 个数据元素。

解答:

答案: $n-i$ 。顺序表删除第 i 个元素后, 原来第 $i+1$ 到第 n 个元素都要依次前移一位, 共有 $n-i$ 个元素需要移动。

32. 小根堆为 [8, 15, 10, 21, 34, 16, 12], 删除关键字 8 并重新建堆时, 关键字比较次数为 _____。

解答:

答案: 3。删除堆顶 8 后, 用最后一个元素 12 补到根, 得到 [12, 15, 10, 21, 34, 16]。先比较两个孩子 15 和 10, 再比较 12 与较小孩子 10, 交换后再比较 12 与其孩子 16, 共 3 次关键字比较。

33. 在广度优先遍历、拓扑排序、求最短路径三种算法中, 可判断有向图是否有环的是 _____。

解答:

答案: 拓扑排序。有向图存在拓扑序列当且仅当它是有向无环图。若拓扑排序过程中仍有顶点未输出但不存在入度为 0 的顶点, 则说明图中存在有向环。

34. n ($n \geq 2$) 个顶点的有向强连通图最少有 _____ 条边。

解答:

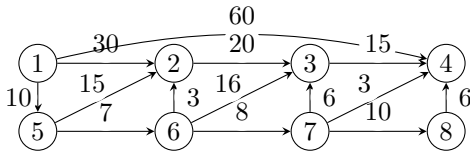
答案: n 。要使每个顶点都能到达其他顶点, 最少可以构成一个包含全部 n 个顶点的有向环, 此时有 n 条有向边, 并且任意两点互相可达。少于 n 条边不可能让所有顶点都处在同一个强连通结构中。

35. 栈 S_1 中操作数自栈底到栈顶为 5, 8, 3, 2, 栈 S_2 中运算符自栈底到栈顶为 *, -, /, 按题给函数调用三次后, 若按整数除法计算, S_1 栈顶为 _____。

解答：
答案：35。第一次弹出运算符 /，用栈顶两个操作数计算 $3/2 = 1$ (整数除法)， S_1 变为 5, 8, 1；第二次弹出 -，计算 $8 - 1 = 7$ ， S_1 变为 5, 7；第三次弹出 *，计算 $5 \times 7 = 35$ ，故最终栈顶为 35。

四、简答题 (3 题，共 20 分)

36. (Dijkstra 算法) 试用 Dijkstra 算法求题图中顶点 1 到其余各顶点的最短路径，写出算法执行过程中各步状态。



所选顶点	U (已确定最短路径的顶点集合)	T (未确定最短路径的顶点集合)	d_2	d_3	d_4	d_5	d_6	d_7	d_8
初始	{1}	{2, 3, 4, 5, 6, 7, 8}	30	∞	60	10	∞	∞	∞

解答：
按题图边权，从顶点 1 出发。用 $d(i)$ 表示当前从 1 到 i 的最短估计距离， ∞ 表示暂不可达。执行过程如下：

步骤	新确定顶点	$d(2)$	$d(3)$	$d(4)$	$d(5)$	$d(6)$	$d(7)$	$d(8)$
0	1	30	∞	60	10	∞	∞	∞
1	5	25	∞	60	10	17	∞	∞
2	6	20	33	60	10	17	25	∞
3	2	20	33	60	10	17	25	∞
4	7	20	31	28	10	17	25	35
5	4	20	31	28	10	17	25	35
6	3	20	31	28	10	17	25	35
7	8	20	31	28	10	17	25	35

因此选点次序为 1, 5, 6, 2, 7, 4, 3, 8，最短距离为

$$(d_2, d_3, d_4, d_5, d_6, d_7, d_8) = (20, 31, 28, 10, 17, 25, 35).$$

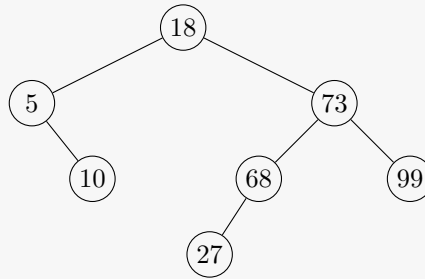
一组对应路径为：1 $\rightarrow$ 5 $\rightarrow$ 6 $\rightarrow$ 2，1 $\rightarrow$ 5 $\rightarrow$ 6 $\rightarrow$ 7 $\rightarrow$ 3，1 $\rightarrow$ 5 $\rightarrow$ 6 $\rightarrow$ 7 $\rightarrow$ 4，1 $\rightarrow$ 5，1 $\rightarrow$ 5 $\rightarrow$ 6，1 $\rightarrow$ 5 $\rightarrow$ 6 $\rightarrow$ 7，1 $\rightarrow$ 5 $\rightarrow$ 6 $\rightarrow$ 7 $\rightarrow$ 8。

37. (二叉搜索树) 给定关键码集合 {18, 73, 5, 10, 68, 99, 27}。

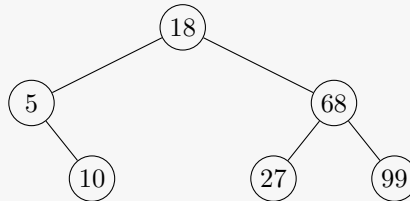
- (1) 画出按记录顺序构造形成的二叉排序 (搜索) 树；
- (2) 画出删除关键码 73 后的二叉排序树；
- (3) 画出对集合 C (未删除 73 前) 查询效率最高的最优二叉排序树，其中每个关键码被查询概率相等。

解答：
顺序插入得到：根为 18；左子树为 5，且 5 的右孩子为 10；右子树根为 73，73 的左孩子为 68、右

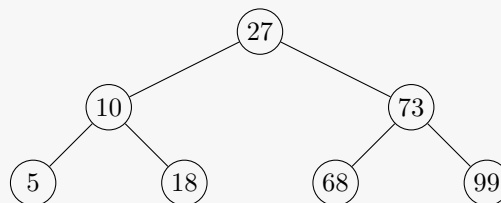
孩子为 99，68 的左孩子为 27。



删除 73 时，可用其左子树最大结点 68 替代，得到右子树根 68，其左孩子为 27、右孩子为 99。



若在原集合 {5, 10, 18, 27, 68, 73, 99} 上等概率查找，最优二叉排序树可取中位数 27 为根，左子树根为 10、右子树根为 73，叶结点为 5, 18, 68, 99。这是高度尽量平衡的二叉搜索树，成功查找平均比较次数最小。



38. (奇偶交换排序) 奇偶交换排序如下：第一趟对所有奇数 i 比较 a_i 与 a_{i+1} ，必要时交换；第二趟对所有偶数 i 比较 a_i 与 a_{i+1} ，必要时交换；如此交替，直到整个序列有序。对序列 {18, 73, 5, 10, 68, 99, 27, 10} 写出前 4 趟结果，说明算法是否稳定，并给出正序、逆序输入时的比较次数和交换次数。

解答：

前 4 趟结果依次为

18, 73, 5, 10, 68, 99, 10, 27,
 18, 5, 73, 10, 68, 10, 99, 27,
 5, 18, 10, 73, 10, 68, 27, 99,
 5, 10, 18, 10, 73, 27, 68, 99.

算法只在相邻元素逆序时交换，相等元素不交换，因此是稳定排序。

若初始序列为正序，比较 $n-1$ 次，交换 0 次。若初始序列为逆序，则每个逆序对最终都要通过一次相邻交换消去，交换次数为

$$\frac{n(n-1)}{2}.$$

按题给停止条件，最后还需一轮无交换检查，比较次数为

$$\frac{n(n-1)}{2} + n - 1.$$

五、算法填空 (2 题，共 20 分)

39. (单链表逆置) 填空完成程序：读入整数序列，用单链表存储，然后将该单链表原地逆置后输出。下面程序使用普通头引用 head，链表结点定义为 Node(data)。

```

class Node:
    def __init__(self, data):
        self.data = data
        self.next = None

def print_list(head):
    p = head
    while p is not None:
        print(p.data, end=" ")
        p = p.next
    print()

def reverse_list(head):
    if head is not None:
        p = head.next
        head.next = _____ # (1)
        while p is not None:
            q = p
            p = _____ # (2)
            q.next = _____ # (3)
            _____ # (4)
        return head

n = int(input())
a = list(map(int, input().split()))
head = Node(a[0])
p = head
for i in range(1, n):
    p.next = _____ # (5)
    p = _____ # (6)

head = reverse_list(head)
print_list(head)

```

解答：

填空答案：(1) None; (2) p.next; (3) head; (4) head = q; (5) Node(a[i]); (6) p.next。

逆置时，先把原头结点变成新链表的尾结点，即令 head.next = None。随后依次取出原链表剩余结点 p，用头插法插入到新链表头部。变量 q 暂存当前结点，p 先后移，随后令 q.next 指向当前新表头，再把 head 更新为 q。

40. (由扩充二叉树先序序列构造二叉树) 输入一棵二叉树的扩充二叉树先序序列，其中 @ 表示空结点。递归构造该二叉树，并输出其中序遍历序列。

例如输入 ABD@@G@@CE@@F@@ 时，对应二叉树的中序遍历为 DGBAECF。

```

class BinaryTreeNode:
    def __init__(self, data):
        self.data = data
        self.left = None
        self.right = None

s = input().strip()
pos = 0

```

```

def build_tree():
    global pos
    ch = s[pos]
    pos += 1
    if ch == "@":
        return _____ # (1)

    root = _____ # (2)
    root.left = _____ # (3)
    root.right = _____ # (4)
    return root

def inorder(root):
    if root is None:
        return

    _____ # (5)
    print(root.data, end=" ")
    _____ # (6)

tree = build_tree()
inorder(tree)

```

解答：

填空答案：(1) None; (2) BinaryTreeNode(ch); (3) build_tree(); (4) build_tree(); (5) inorder(root.left); (6) inorder(root.right)。

扩充先序序列的读取规则是“根、左子树、右子树”。遇到 @ 表示当前子树为空，直接返回 None；否则建立当前根结点，并继续递归构造左、右子树。中序遍历顺序为“左子树、根、右子树”。

数据结构与算法 B

common-23 春-期末

一、选择题 (30 分, 每小题 2 分)

1. 下列叙述中正确的是 ()。

- (A) 散列是一种基于索引的逻辑结构
- (B) 基于顺序表实现的逻辑结构属于线性结构
- (C) 数据结构设计影响算法效率, 逻辑结构起到了决定作用
- (D) 一个逻辑结构可以有多种类型的存储结构, 且不同类型的存储结构会直接影响到数据处理效率

解答:

选 D。逻辑结构描述数据元素之间的关系, 存储结构描述这种关系在计算机中的实现方式。同一种逻辑结构可以采用顺序、链式、索引、散列等不同存储方式, 不同存储方式会直接影响查找、插入、删除等操作效率。

2. 在广度优先搜索算法中, 一般使用什么辅助数据结构? ()

- (A) 队列
- (B) 栈
- (C) 树
- (D) 散列

解答:

选 A。广度优先搜索按照“先发现的顶点先扩展”的顺序进行, 符合队列的先进先出特性。

3. 若某线性表实现采取链式存储, 那么该线性表中结点的存储地址 ()。

- (A) 一定不连续
- (B) 既可连续亦可不连续
- (C) 一定连续
- (D) 与头结点存储地址保持连续

解答:

选 B。链式存储通过引用或链接表示结点之间的逻辑关系, 结点的物理地址不要求连续; 实际分配时也可能碰巧连续, 因此应表述为既可连续也可不连续。

4. 若某线性表的常用操作是在表尾插入或删除元素, 则时间开销最小的存储方式是 ()。

- (A) 单链表
- (B) 仅有头引用的单循环链表
- (C) 顺序表
- (D) 仅有尾引用的单循环链表

解答:

选 C。顺序表在表尾插入、删除元素通常只需修改表长, 时间为 $O(1)$ (不考虑扩容)。单链表或单循环链表若要删除尾结点, 通常需要寻找尾结点前驱, 时间为 $O(n)$ 。

5. 给定后缀表达式 (逆波兰式) $ab+-c*d-$, 其对应的中缀表达式是 ()。

- (A) $a-b-c*d$
- (B) $-(a+b)*c-d$
- (C) $a+b*c-d$
- (D) $(a+b)*(-c-d)$

解答：

选 B。先由 $ab+$ 得到 $(a+b)$ ，随后的一元负号得到 $-(a+b)$ ；再与 c 相乘，最后减去 d ，即 $-(a+b)c-d$ 。

6. 今有一空栈 S ，基于待进栈的数据元素序列 a, b, c, d, e, f ，依次进行进栈、进栈、出栈、进栈、进栈、出栈的一系列操作，上述操作完成以后，栈 S 的栈顶元素为 ()。

- (A) f (B) c (C) a (D) b

解答：

选 B。操作过程为： a 入栈、 b 入栈、弹出 b 、 c 入栈、 d 入栈、弹出 d ，此时栈中自底向顶为 a, c ，栈顶为 c 。

7. 对于带头结点的单链表，其表头为 `head`，则判定空表的条件是 ()。

- (A) `head is None` (C) `head.next is head`
(B) `head is not None` (D) `head.next is None`

解答：

选 D。带头结点的单链表即使为空也保留头结点；在 Python 风格的链表表示中，空表时头结点的 `next` 引用为空，因此条件为 `head.next is None`。

8. 以下典型排序算法中，具有稳定排序特性的是 ()。

- (A) 冒泡排序 (Bubble Sort)
(B) 直接选择排序 (Selection Sort)
(C) 快速排序 (Quick Sort)
(D) 希尔排序 (Shell Sort)

解答：

选 A。冒泡排序只交换逆序的相邻元素，相等元素通常不会交换，所以稳定；直接选择排序、快速排序和希尔排序一般都不稳定。

9. 以下关键字序列中，可以有效构成一个大根堆 (即最大值堆，最大值在堆顶) 的序列是 ()。

- (A) 5, 8, 1, 3, 9, 6, 2, 7
(B) 9, 8, 1, 7, 5, 6, 2, 3
(C) 9, 8, 6, 3, 5, 1, 2, 7
(D) 9, 8, 6, 7, 5, 1, 2, 3

解答：

选 D。按顺序存储的堆满足每个父结点关键字都不小于其左右孩子。D 中 $9 \geq 8, 6, 8 \geq 7, 5, 6 \geq 1, 2, 7 \geq 3$ ，满足大根堆性质。

10. 以下典型排序算法中，内存开销最大的是 ()。

- (A) 冒泡排序 (C) 归并排序
(B) 快速排序 (D) 堆排序

解答：

选 C。归并排序通常需要 $O(n)$ 的辅助数组；快速排序主要使用递归栈，平均 $O(\log n)$ ；冒泡排序和

堆排序通常为 $O(1)$ 额外空间。

11. 排序算法往往依赖于对待排序元素序列的多趟比较/移动操作 (即执行多轮循环), 第一趟结束后, 任一元素都无法确定其最终排序位置的算法是 ()。

- (A) 选择排序 (C) 冒泡排序
(B) 快速排序 (D) 插入排序

解答:

选 D。选择排序第一趟可确定最小 (或最大) 元素的位置; 快速排序一趟划分后枢轴到达最终位置; 冒泡排序第一趟通常可确定一个最大 (或最小) 元素的位置。插入排序第一趟只保证局部有序, 元素最终位置不能确定。

12. 考察以下基于单链表的操作, 相较于顺序表实现, 带来更高时间复杂度的操作是 ()。

- (A) 合并两个有序线性表, 并保持合成后的线性表依然有序
(B) 交换第一个元素与第二个元素的值
(C) 查找某一元素值是否在线性表中出现
(D) 输出第 i 个 ($0 \leq i < n$, n 为元素个数) 元素

解答:

选 D。顺序表可按下标随机访问, 第 i 个元素访问为 $O(1)$; 单链表必须从头引用开始逐结点后移, 访问第 i 个元素为 $O(i)$, 时间复杂度更高。

13. 已知一个整型数组中的元素值依次为 {19, 20, 50, 61, 73, 85, 11, 39}, 采用某种排序算法, 在多趟比较/移动操作后, 依次得到如下中间结果:

- (1) 19 20 11 39 73 85 50 61,
(2) 11 20 19 39 50 61 73 85,
(3) 11 19 20 39 50 61 73 85.

请问, 上述过程使用的排序算法是 () 算法。

- (A) 冒泡排序 (B) 插入排序 (C) 希尔排序 (D) 归并排序

解答:

选 C。第一趟结果类似按增量 4 分组排序: (19, 73)、(20, 85)、(50, 11)、(61, 39) 分别调整后得到 19, 20, 11, 39, 73, 85, 50, 61; 之后增量减小继续排序, 符合希尔排序过程。

14. 今有一非连通无向图, 共有 36 条边, 该图至少有 () 个顶点。

- (A) 8 (B) 9 (C) 10 (D) 11

解答:

选 C。若 n 个顶点的无向图非连通, 则边数最多时可由 $n - 1$ 个顶点组成完全图, 另一个顶点孤立, 最多边数为 $\binom{n-1}{2}$ 。令 $\binom{n-1}{2} \geq 36$, 最小满足的是 $n - 1 = 9$, 故 $n = 10$ 。

15. 令 $G = (V, E)$ 是一个无向图, 若 G 中任何两个顶点之间均存在唯一的简单路径相连, 则下面说法中错误的是 ()。

- (A) 图 G 中添加任何一条边, 不一定造成图包含一个环

- (B) 图 G 中移除任意一条边得到的图均不连通
- (C) 图 G 的逻辑结构实际上退化为树结构
- (D) 图 G 中边的数目一定等于顶点数目减 1

解答：

选 A。任意两点之间存在唯一简单路径说明 G 是一棵树。树中任意两个顶点之间已有一条唯一简单路径，再添加一条边会和原路径构成一个环，因此 A 错误；B、C、D 都是树的性质。

二、判断题 (10 分，每小题 1 分；对填写 Y，错填写 N)

16. 按照前序、中序、后序方式周游一棵二叉树，分别得到不同的结点周游序列，然而三种不同的周游序列中，叶子结点都将以相同的顺序出现。()

解答：

答案：Y。无论采用前序、中序还是后序遍历，叶子结点之间的相对次序都与它们在树中的从左到右次序一致，因此叶子结点在三种序列中的相对顺序相同。

17. 构建一个含 N 个结点的 (二叉) 最小值堆，时间效率最优情况下的时间复杂度大 O 表示为 $O(N \log N)$ 。()

解答：

答案：N。若采用自底向上的建堆方法，建堆时间复杂度为 $O(N)$ ，优于逐个插入造成的 $O(N \log N)$ 。

18. 对任意一个连通的无向图，如果存在一个环，且这个环中的某一条边的权值不小于该环中任意一个其它的边的权值，那么这条边一定不会是该无向图的最小生成树中的边。()

解答：

答案：N。环性质要求“严格大于”环中其它边的边一定不在任何最小生成树中；若只是“不小于”，可能与其它边权值相等，此时该边仍可能出现在某棵最小生成树中。

19. 通过树的周游可以求得树的高度，若采取深度优先遍历方式设计求解树高度问题的算法，算法空间复杂度大 O 表示为 $O(\text{树的高度})$ 。()

解答：

答案：Y。深度优先遍历递归求树高时，递归栈深度等于当前根到结点的路径长度，最大为树高，因此辅助空间为 $O(h)$ 。

20. 树可以等价转化二叉树，树的先序遍历序列与其相应的二叉树的前序遍历序列相同。()

解答：

答案：Y。树转化为二叉树通常采用“左孩子右兄弟”表示，树的先根遍历与转换后二叉树的先序遍历访问顺序一致。

21. 如果一个连通无向图 G 中所有边的权值均不同，则 G 具有唯一的最小生成树。()

解答：

答案：Y。边权互异时，Kruskal 或 Prim 每一步对最小安全边的选择不会出现并列，从而最小生成树唯一。

22. 求解最小生成树问题时，Kruskal 算法更适用于稀疏图，Prim 算法更适用于稠密图。()

解答:

答案: Y。Kruskal 主要按边排序和并查集合并, 适合边数较少的稀疏图; Prim 使用邻接矩阵时可在 $O(n^2)$ 内完成, 常用于稠密图。

23. 使用线性探测法处理散列表碰撞问题, 若表中仍有空槽 (空单元), 插入操作一定成功。()

解答:

答案: Y。线性探测按固定步长 1 循环检查表中位置, 只要探测过程完整且散列表仍有空槽, 最终一定能找到空槽插入。

24. 从链表中删除某个指定值的结点, 其时间复杂度是 $O(1)$ 。()

解答:

答案: N。若只给定要删除的值, 必须先从链表中查找该值所在结点及其前驱, 查找过程平均和最坏都可能为 $O(n)$ 。

25. 弗洛伊德 (Floyd) 算法是一种求解有向和无向图最短路径的动态规划算法, 但该算法的局限性是无法正确求解带有负权值边的图的最短路径。()

解答:

答案: N。Floyd 算法可以处理负权边, 但不能处理存在可达负权回路的情形; 只要不存在负权回路, 负权边并不妨碍正确求解最短路径。

三、填空题 (20 分, 每题 2 分)

26. 定义二叉树中一个结点的度数为其子结点的个数。现有一棵结点总数为 101 的二叉树, 其中度数为 1 的结点有 30 个, 则度数为 0 结点有 _____ 个。

解答:

答案: 36。设度为 0, 1, 2 的结点数分别为 n_0, n_1, n_2 。二叉树有性质 $n_0 = n_2 + 1$, 且 $n_0 + n_1 + n_2 = 101$, 已知 $n_1 = 30$, 所以 $n_0 + n_2 = 71$, 解得 $n_2 = 35, n_0 = 36$ 。

27. 定义完全二叉树的根结点所在层为第一层, 如果一个二叉树的第六层有 23 个叶结点, 则它的总结点数量可能有 _____ 个。(请填写所有 3 个可能的结点数。)

解答:

答案: 54, 80, 81。若树高为 6, 前 5 层共有 $2^5 - 1 = 31$ 个结点, 第六层有 23 个叶结点, 总数为 $31 + 23 = 54$ 。若树高为 7, 第六层满 32 个结点, 其中有孩子的第六层结点数为 $32 - 23 = 9$; 第七层结点数可为 17 或 18, 总数为 $31 + 32 + 17 = 80$ 或 $31 + 32 + 18 = 81$ 。

28. 对于初始排序码序列 (51, 41, 31, 21, 61, 71, 81, 11, 91), 第 1 趟快速排序 (以第一个数字为 pivot 轴值) 的结果是: _____。

解答:

答案: (11, 41, 31, 21, 51, 71, 81, 61, 91)。以 51 为枢轴, 从右向左找到小于 51 的 11 放到左端空位, 再从左向右找到大于 51 的 61 放到右端空位, 最后左右下标相遇, 将 51 放入中间位置, 得到该划分结果。

29. 如果输入序列是已经正序, 在 (改进) 冒泡排序、直接插入排序和直接选择排序算法中, _____ 算法最慢结束。

解答：

答案：直接选择排序。改进冒泡排序在一趟扫描无交换后即可结束，为 $O(n)$ ；直接插入排序对正序序列也近似 $O(n)$ ；直接选择排序无论初始序列如何都要进行多轮选择比较，仍为 $O(n^2)$ 。

30. 已知某二叉树的先根周游序列为 $\{A, B, D, E, C, F, G\}$ ，中根周游序列为 $\{D, B, E, A, C, G, F\}$ ，则该二叉树的后根次序周游序列为 $\{\text{_____}\}$ 。

解答：

答案： D, E, B, G, F, C, A 。先序首元素 A 为根，中序以 A 分成左子树 D, B, E 和右子树 C, G, F 。左子树根为 B ，后序为 D, E, B ；右子树根为 C ，其右子树根为 F 且 G 为 F 的左孩子，后序为 G, F, C ；合并得 D, E, B, G, F, C, A 。

31. 使用栈计算后缀表达式 (操作数均为一位数) $1\ 2\ 3\ +\ 4\ *\ 5\ +\ 3\ +\ -$ ，当扫描到第二个 $+$ 号但还未对该 $+$ 号进行运算时，栈的内容 (以栈底到栈顶从左往右的顺序书写) 为 _____。

解答：

答案： $1, 20, 5$ 。扫描 $1, 2, 3, +$ 后， $2 + 3 = 5$ ，栈为 $1, 5$ ；再扫描 $4, *$ 后， $5 \times 4 = 20$ ，栈为 $1, 20$ ；继续扫描操作数 5 后遇到第二个加号，尚未计算前栈为 $1, 20, 5$ 。

32. 51 个顶点的连通图 G 有 50 条边，其中权值为 $1, 2, 3, 4, 5, 6, 7, 8, 9, 10$ 的边各 5 条，则连通图 G 的最小生成树各边的权值之和为 _____。

解答：

答案：275。51 个顶点的连通图若只有 50 条边，则它本身就是一棵树；树的最小生成树就是自身。因此权值和为 $5(1 + 2 + \dots + 10) = 5 \times 55 = 275$ 。

33. 包含 n 个顶点无向图的邻接表存储结构中，所有顶点的边表中最多有 _____ 个结点。具有 n 个顶点的有向图，一个顶点入度和出度之和最大值不超过 _____。

解答：

答案： $n(n-1)$ ； $2(n-1)$ 。无向简单图最多有 $\binom{n}{2}$ 条边，邻接表中每条无向边出现两次，因此最多有 $n(n-1)$ 个边表结点。有向图中一个顶点最多向其它 $n-1$ 个顶点发出边，也最多从其它 $n-1$ 个顶点接收边，所以入度与出度之和最大为 $2(n-1)$ 。

34. 给定一个长度为 7 的空散列表，采用双散列法解决冲突，两个散列函数分别为 $h_1(key) = key \% 7$ ， $h_2(key) = key \% 5 + 1$ 。请向散列表依次插入关键字为 30, 58, 65 的集合元素，插入完成后 65 在散列表中存储地址为 _____。

解答：

答案：3。 $30 \% 7 = 2$ ，先放入地址 2； $58 \% 7 = 2$ 发生冲突，步长 $58 \% 5 + 1 = 4$ ，放入 $(2 + 4) \% 7 = 6$ ； $65 \% 7 = 2$ 也冲突，步长 $65 \% 5 + 1 = 1$ ，下一地址为 3，故 65 存在地址 3。

35. 阅读 Python 算法 ABC，回答问题。

```
from math import isqrt

def ABC(n):
    m = isqrt(n)
    for k in range(2, m + 1):
        if n % k == 0:
```

```
        return 0
    return 1
```

- (1) 算法的功能是：_____。
- (2) 算法的时间复杂度是 $O(\text{_____})$ 。

解答：

答案：(1) 判断 n 是否为素数 (若没有 2 到 $\lfloor \sqrt{n} \rfloor$ 范围内的因子则返回 1，否则返回 0)；(2) $\sqrt{n}$ 。Python 中 `isqrt(n)` 返回 $\lfloor \sqrt{n} \rfloor$ ，循环最多检查从 2 到 $\lfloor \sqrt{n} \rfloor$ 的整数因子，因此时间复杂度为 $O(\sqrt{n})$ 。

四、简答题 (3 题，共 14 分)

36. (字符串匹配与 KMP) 字符串匹配算法从长度为 n 的文本串 S 中查找长度为 m 的模式串 P 的首次出现位置。

- 字符串匹配的朴素算法使用暴力搜索，大致过程如下：对于 P 在 S 中可能出现的 $n - m + 1$ 个位置，比对此位置时 P 和 S 中对应子串是否相等。其时间复杂度 $O((n - m + 1)m)$ 。请举例说明算法时间复杂度一种最坏情况 (注：例子中请只出现 0 和 1 两种字符)。
- 已知字符串 S 为 `abaabaabacacaabaabcc`，模式串 t 为 `abaabc`。采用 KMP 算法进行查找，请写出 `next` 数组。
- 说明在上述 (2) 步骤中，第一次出现不匹配 ($s[i] \neq t[j]$) 时， $i = j = 5$ ；则下次比对时， i 和 j 的值分别是？

解答：

- 可取文本串 $S = 00 \dots 00$ ，模式串 $P = 00 \dots 01$ (前 $m - 1$ 位为 0，最后一位为 1)。每个可能起始位置都会先匹配很多个 0，最后才因 1 不匹配而失败，从而达到 $O((n - m + 1)m)$ 的最坏量级。
- 采用下标从 0 开始且 `next[0] = -1` 的常见定义， $t = \text{abaabc}$ 的 `next` 数组为

`[-1, 0, 0, 1, 1, 2].`

- 第一次不匹配在 $i = j = 5$ ，失配后 i 不回退， j 回退到 `next[5]`，所以下次比对时 $i = 5, j = 2$ 。

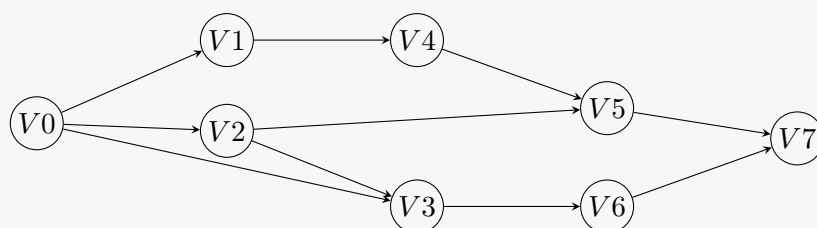
37. (AOV 网络与拓扑排序) 有八项活动，每项活动标记为 $V_n (0 \leq n \leq 7)$ ，每项活动要求的前驱如下：

活动	V_0	V_1	V_2	V_3	V_4	V_5	V_6	V_7
前驱	无前驱	V_0	V_0	V_0, V_2	V_1	V_2, V_4	V_3	V_5, V_6

试画出相应的 AOV 网络，并给出一个拓扑排序序列，如存在多种，则按照编号从小到大排序，输出最小的一种。

解答：

由前驱关系得到边： $V_0 \rightarrow V_1, V_2, V_3$ ， $V_1 \rightarrow V_4$ ， $V_2 \rightarrow V_3, V_5$ ， $V_4 \rightarrow V_5$ ， $V_3 \rightarrow V_6$ ， $V_5 \rightarrow V_7$ ， $V_6 \rightarrow V_7$ 。一个可画作如下的 AOV 网络：



每次在当前入度为 0 的顶点中选编号最小者，得到拓扑序列

$V_0, V_1, V_2, V_3, V_4, V_5, V_6, V_7.$

38. (Huffman 树与 Huffman 编码) 简要回答下列 Huffman 树以及 Huffman 编码的相关问题。

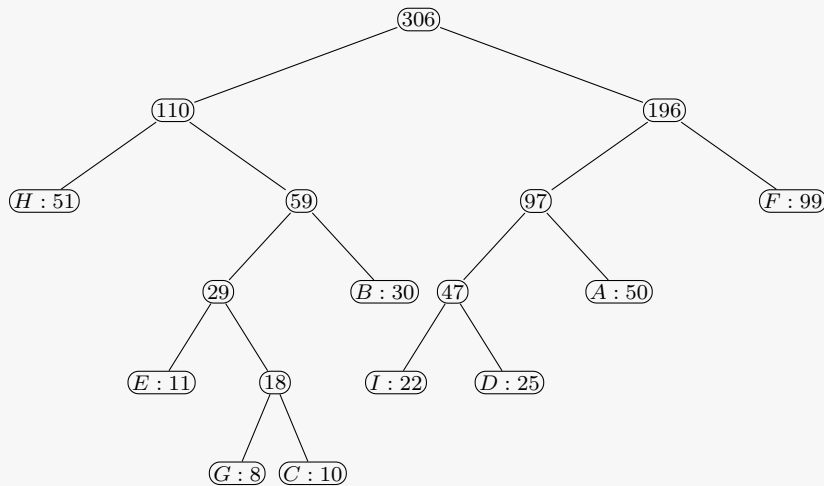
- (1) 假定需要编码的字符个数为 m ，哈夫曼树构造后结点总数多少？简要阐述理由。
- (2) 假定报文中各字符及其出现次数为：A(50), B(30), C(10), D(25), E(11), F(99), G(8), H(51), I(22)。试画出对应的 Huffman 树 (严格按照左小右大构造)，并给出各个字符的 Huffman 编码 (左分支编码 0，右分支编码 1)。

解答：

- (1) 结点总数为 $2m - 1$ 。Huffman 树是具有 m 个叶子结点的满二叉树，每次合并两个结点产生一个新内部结点，总共合并 $m - 1$ 次，因此内部结点数为 $m - 1$ ，总结点数为 $m + (m - 1) = 2m - 1$ 。
- (2) 按权值从小到大反复合并，过程为：8 + 10 = 18，11 + 18 = 29，22 + 25 = 47，29 + 30 = 59，47 + 50 = 97，51 + 59 = 110，97 + 99 = 196，110 + 196 = 306。严格左小右大时，编码为：

字符	A	B	C	D	E	F	G	H	I
编码	101	011	01011	1001	0100	11	01010	00	1000

对应树可表示为：



五、算法填空 (4 题，共 26 分)

39. (有序链表删除重复元素) 请填空完成下列 Python 程序：读入一个从小到大排好序的整数序列到链表，然后在链表中删除重复的元素，使得重复的元素只保留 1 个，然后将整个链表内容输出。

输入：第一行是正整数 $n(n < 80)$ ，第二行是 n 个整数。输入样例为 9 和 1 2 2 2 3 3 4 4 6，输出样例为 1 2 3 4 6。

```

class Node:
    def __init__(self, data):
        self.data = data
        self.next = None

n = int(input())
a = list(map(int, input().split()))
head = Node(a[0])
p = head
for x in a[1:]:
    p.next = Node(_____) # (1)
    p = _____ # (2)

p = head
while p is not None:
    while _____ and p.data == p.next.data: # (3)
        _____ # (4)
    p = p.next
    
```

```
p = head
while p is not None:
    print(p.data, end=" ")
    ----- # (5)
```

解答:

空白处依次为:

- (1) x
- (2) p.next
- (3) p.next is not None
- (4) p.next = p.next.next
- (5) p = p.next

建链时, `p.next` 是新建结点, 随后令 `p` 后移到这个新结点。删除重复值时, 由于序列已有序, 重复元素必相邻; 只要 `p.next` 存在且数据相等, 就让 `p.next` 越过重复结点。输出时每打印一个结点后将 `p` 后移。

40. (堆排序) 为实现对一组数据根据排序码进行自小到大排序, 请填空完成下列 Python 堆排序程序 (仅含关键部分代码)。每条记录用字典表示, 其中 `record["key"]` 为排序码。程序中建立的堆是大根堆 (即最大值堆, 最大元素在堆顶)。

```
def sift(records, i, n):
    temp = records[i]
    child = ----- # (1)
    while child < n:
        if ----- and records[child]["key"] < records[child + 1]["key"]: # (2)
            child += 1
        if temp["key"] < records[child]["key"]:
            ----- # (3)
            i = child
            child = ----- # (4)
        else:
            break
    records[i] = temp

def heap_sort(records):
    n = len(records)
    for i in range(n // 2 - 1, -1, -1):
        ----- # (5)
    for i in range(n - 1, 0, -1):
        records[0], records[i] = records[i], records[0]
        ----- # (6)
```

解答:

空白处依次为:

- (1) `2 * i + 1`
- (2) `child < n - 1`
- (3) `records[i] = records[child]`
- (4) `2 * i + 1`
- (5) `sift(records, i, n)`
- (6) `sift(records, 0, i)`

在 0 下标存储的完全二叉树中，结点 i 的左孩子为 $2i + 1$ ，右孩子为 $2i + 2$ 。调整大根堆时应先选择较大的孩子，再将较大孩子上移。每次把堆顶最大元素交换到末尾后，只需对剩余区间 $[0, i)$ 再调整堆。Python 的 `range(n // 2 - 1, -1, -1)` 会从最后一个非叶结点一直枚举到根结点 0。

41. (AOV 网的拓扑排序程序) 以下 Python 代码实现了对 AOV 网的拓扑排序。设 `graph[i]` 存放从顶点 i 出发的所有邻接点编号，请阅读代码并补全空缺。

```
def find_indegree(graph):
    indegree = [0] * len(graph)
    for i in range(len(graph)):
        for v in graph[i]:
            # (1)
            -----
    return indegree

def topo_sort(graph):
    indegree = find_indegree(graph)
    stack = []
    topo = []
    for i in range(len(graph)):
        if indegree[i] == 0:
            # (2)
            -----

    while -----:
        # (3)
        j = stack.pop()
        topo.append(j)
        for k in -----:
            # (4)
            indegree[k] = -----
            # (5)
            if indegree[k] == 0:
                # (6)
                -----

    if -----:
        # (7)
        print("The aov network has a cycle")
        return None
    return topo
```

解答：

空白处依次为：

- (1) `indegree[v] += 1`
- (2) `stack.append(i)`
- (3) `stack`
- (4) `graph[j]`
- (5) `indegree[k] - 1`
- (6) `stack.append(k)`
- (7) `len(topo) < len(graph)`

求入度时，每扫描到一条指向顶点 v 的边，就将该顶点入度加一。拓扑排序过程中，栈中保存当前入度为 0 的顶点；弹出一个顶点并输出后，需要把它所有后继顶点的入度减 1。若最终输出顶点数小于图中顶点数，说明仍有顶点未输出，AOV 网中存在环。

42. (无向图连通性与回路判断) 请填空完成下列 Python 程序：给定一个无向图，判断是否连通，是否有回路。图用邻接矩阵 G 表示，`G[v][u] == 1` 表示顶点 v 与顶点 u 相邻。

```
def dfs_connection(G, visited, v):
    visited[v] = True
```

```

total = 1
for u in range(len(G)):
    if G[v][u] and not visited[u]:
        total += dfs_connection(G, visited, u)
return total

def dfs_loop(G, visited, v, parent):
    visited[v] = True
    for u in range(len(G)):
        if G[v][u]:
            if not visited[u]:
                if _____:          # (1)
                    return True
            else:
                if _____:          # (2)
                    return True
    return False

n, m = map(int, input().split())
G = [[0] * n for _ in range(n)]
for _ in range(m):
    a, b = map(int, input().split())
    G[a][b] = G[b][a] = 1

visited = [False] * n
total = dfs_connection(G, visited, 0)
if _____:                        # (3)
    print("connected:yes")
else:
    print("connected:no")

visited = [False] * n
loop_found = False
for i in range(n):
    if _____:                    # (4)
        if dfs_loop(G, visited, i, -1):
            loop_found = True
            break

if loop_found:
    print("loop:yes")
else:
    print("loop:no")

```

解答:

空白处依次为:

- (1) `dfs_loop(G, visited, u, v)`
- (2) `u != parent`
- (3) `total == n`
- (4) `not visited[i]`

第一次 DFS 后, 若访问到的顶点总数等于 n , 图连通。判断无向图是否有环时, 从 v 访问到未访问的邻接点 u 就递归; 若邻接点已经访问过且不是 DFS 树上 v 的父结点 `parent`, 则发现回边, 说明存在回路。外层循环能处理非连通图的每个连通分量。

数据结构与算法 B

common-24 春-期末

一、选择题 (30 分, 每小题 2 分)

1. 设栈的输入序列为 1, 2, 3, 4, 5, 下列序列中不可能是其出栈序列的是 ()。

- | | |
|-----------|-----------|
| (A) 23415 | (C) 23145 |
| (B) 54132 | (D) 15432 |

解答:

选 B。若 5 最先出栈, 则 1, 2, 3, 4 已依次在栈内, 之后必须按 4, 3, 2, 1 的相对次序出栈, 不可能出现 5, 4, 1, 3, 2。

2. 对线性表进行二分法查找, 其前提条件是 ()。

- | | |
|--------------------------------|--------------------------------|
| (A) 线性表以链接方式存储, 并且按关键码值排序 | (C) 线性表以顺序方式存储, 并且按关键码值排序 |
| (B) 线性表以链接方式存储, 并且按关键码值的检索频率排序 | (D) 线性表以顺序方式存储, 并且按关键码值的检索频率排序 |

解答:

选 C。二分查找需要能随机访问中间元素, 同时要求关键码有序。

3. 对于二叉树中的结点 N , 若其左子树高度为 h_1 , 右子树高度为 h_2 , 则以 N 为根结点的子树高度为 ()。

- | | |
|----------------------|--------------------------|
| (A) $\max(h_1, h_2)$ | (C) $\max(h_1, h_2) + 1$ |
| (B) $\min(h_1, h_2)$ | (D) $\min(h_1, h_2) + 1$ |

解答:

选 C。根结点所在层还需计入高度。

4. 在插入排序算法中, 如果待排序序列已经是有序的, 则插入排序算法的时间复杂度为 ()。

- | | |
|-------------------|------------|
| (A) $O(n^2)$ | (C) $O(n)$ |
| (B) $O(n \log n)$ | (D) $O(1)$ |

解答:

选 C。此时每一趟只需一次比较即可确定无需移动, 总时间为线性阶。

5. 下列概念中属于存储结构的是 ()。

- | | |
|---------|--------|
| (A) 线性表 | (C) 栈 |
| (B) 链表 | (D) 队列 |

解答:

选 B。链表是链式存储结构, 其余通常为逻辑结构或抽象数据类型。

6. 在二分查找算法中, 每次比较后都将搜索范围缩小一半。若要在长度为 n 的数组中查找一个元素, 最坏情况下需要比较多少次? ()

- (A) n (C) $\lfloor \log_2 n \rfloor$
 (B) $\lfloor n/2 \rfloor$ (D) $\lfloor \log_2 n \rfloor + 1$

解答：

选 D。二分查找判定树高度为 $\lfloor \log_2 n \rfloor + 1$ 。

7. 在一个具有 n 个结点的有序单链表中插入一个新结点并仍保持其有序，平均时间复杂度为 ()。

- (A) $O(n \log n)$ (C) $O(n)$
 (B) $O(1)$ (D) $O(n^2)$

解答：

选 C。需要顺链查找插入位置，平均需扫描线性数量的结点。

8. 设有 4 棵结点关键码均为整数的二叉树，若其中序遍历序列分别如下，可能是二叉搜索树的是 ()。

- (A) 5, 3, 1, 2, 4 (C) 5, 3, 4, 2, 1
 (B) 1, 2, 3, 4, 5 (D) 1, 3, 5, 4, 2

解答：

选 B。二叉搜索树的中序遍历序列应为非递减序列。

9. 线性表有 $2n$ ($n > 100$) 个元素，以下操作中，哪一项在单链表上实现比在顺序表上实现效率更高? ()

- (A) 在表中最后一个元素的后面插入一个新元素 (C) 交换表中第 i 个元素和第 $n + i$ 个元素的值
 (B) 顺序输出表中的前 i 个元素 ($i = 0, \dots, n - 1$)
 (D) 删除表中第 1 个元素

解答：

选 D。顺序表删除首元素需移动大量后继元素；单链表删除首结点只需修改头引用。

10. 用数组 $O[n]$ 实现循环队列。设队头的前一个元素下标为 f ，队尾下标为 r ，则队列中元素总数为 ()。

- (A) $r - f$ (C) $n + r - f$
 (B) $(n + f - r) \bmod n$ (D) $(n + r - f) \bmod n$

解答：

选 D。循环队列中从 f 的后继走到 r 的元素个数为 $(r - f + n) \bmod n$ 。

11. 一个拥有 21 条边的非连通无向图，至少包含 () 个顶点。

- (A) 5 (C) 7
 (B) 6 (D) 8

解答：

选 D。 v 个顶点的非连通无向简单图边数至多为 $\binom{v-1}{2}$ 。当 $v = 7$ 时最多 15 条边，不足 21；当 $v = 8$ 时最多 21 条边。

12. 若使用分治算法解决某问题，每次把规模为 n 的问题分解为 2 个规模为 $n/2$ 的子问题，并用 $O(n)$ 时间合并两个子问题结果，则总时间复杂度为 ()。

- (A) $O(n)$
- (B) $O(n \log n)$
- (C) $O(n^2)$
- (D) $O(n^2 \log n)$

解答：

选 B。递推式为 $T(n) = 2T(n/2) + O(n)$ ，由主定理得 $T(n) = O(n \log n)$ 。

13. 有 n 个顶点、 m 条边的无向图，下列说法中错误的是 ()。

- (A) 采用邻接表存储，空间复杂度为 $O(n + m)$ 间复杂度为 $O(n + m)$
- (B) 若为稠密图，更倾向于采用邻接矩阵存储 (D) 若为稀疏图，深度优先周游的时间复杂度低于
- (C) 采用邻接表存储，删除一个顶点的最坏情况时 广度优先周游的时间复杂度

解答：

选 D。邻接表存储下 DFS 与 BFS 的时间复杂度均为 $O(n + m)$ 。

14. 利用栈将表达式 $3 * 2^{(4 + 2 * 2 - 6 * 3)} - 5$ (其中 $\wedge$ 为乘幂) 转换为后缀表达式。扫描到 6 时，运算符栈为 ()。

- (A) $*^{(+ -}$
- (B) $*^{(}$
- (C) $*^{(+$
- (D) $*^{(-$

解答：

选 D。扫描到 6 前，括号内的 $+$ 与 $*$ 已因遇到 $-$ 出栈，栈中自底向上为 $*^{(-$ 。

15. 定义一棵没有 1 度结点的二叉树为满二叉树。对于一棵包含 k 个结点的满二叉树，其中叶子结点的个数为 ()。

- (A) $\lfloor k/2 \rfloor$ (C) $\lfloor k/2 \rfloor + 1$
- (B) $\lfloor k/2 \rfloor - 1$ (D) 以上三个都有可能

解答：

选 C。满二叉树中若分支结点数为 m ，叶子数为 $m + 1$ ，总数 $k = 2m + 1$ ，故叶子数为 $(k + 1)/2 = \lfloor k/2 \rfloor + 1$ 。

二、判断题 (10 分，每小题 1 分；对填 Y，错填 N)

16. 分治法和动态规划法都适用于将问题分解为规模较小的子问题的思想。()

解答：

答案：Y。分治法和动态规划法都先把原问题分解成子问题。区别在于：分治法通常处理相互独立的子问题，动态规划则特别适合有重叠子问题且具有最优子结构的问题。

17. 在链表中查找和删除一个元素的时间复杂度都是 $O(1)$ 。()

解答：

答案：N。若已知待删结点的前驱，链表删除可通过修改引用在 $O(1)$ 时间完成；但查找某个元素通

常必须从头结点开始顺序扫描，时间复杂度为 $O(n)$ 。所以“查找和删除都是 $O(1)$ ”不正确。

18. 相比于顺序存储，完全二叉树更适合用链式表示存储。()

解答：

答案：N。完全二叉树结点排列紧凑，按层序编号后父子关系可由下标直接计算，因此顺序存储非常适合。链式存储反而需要额外引用域空间，通常不是完全二叉树的首选。

19. 通常不能仅通过一棵二叉树的前序遍历结点序列和后序遍历结点序列确定这棵二叉树的完整结构。()

解答：

答案：Y。前序和后序都能确定根的位置，但不能区分某个单孩子结点的孩子到底是左孩子还是右孩子。因此一般情况下仅凭前序和后序不能唯一确定二叉树结构。

20. 二叉搜索树一定是满二叉树。()

解答：

答案：N。二叉搜索树只要求左子树关键字小于根、右子树关键字大于根，并不要求每个分支结点都有两个孩子。按有序序列插入时，二叉搜索树甚至可能退化成单链。

21. 归并排序算法一定比简单插入排序算法的执行效率高。()

解答：

答案：N。归并排序的渐进复杂度为 $O(n \log n)$ ，插入排序最坏为 $O(n^2)$ ；但当数据规模很小或原序列接近有序时，插入排序可能因常数小、移动少而更快。因此不能说归并排序“一定”更高效。

22. 在待排序元素为正序情况下，直接插入排序可能比快速排序的时间复杂度更小。()

解答：

答案：Y。直接插入排序在正序输入时只需相邻比较而几乎不移动，时间复杂度为 $O(n)$ 。若快速排序总选第一个元素作枢轴，正序输入会导致极不平衡划分，最坏可达 $O(n^2)$ 。

23. 一个有 n 个结点的无向图，最少有一个连通分量。()

解答：

答案：Y。只要图中存在顶点，每个顶点至少属于某个连通分量；即使没有任何边，每个孤立顶点也是一个连通分量。因此有 n 个结点的无向图至少有一个连通分量。

24. 图的遍历算法 (如深度优先搜索 DFS 和广度优先搜索 BFS) 都只能用于无向图。()

解答：

答案：N。DFS 和 BFS 既可用于无向图，也可用于有向图。对于有向图，遍历时只沿有向边允许的方向访问相邻顶点即可。

25. 若连通无向图的边权值互不相同，其最小生成树只有一个。()

解答：

答案：Y。当所有边权互不相同，每次按割性质选择的最轻边都是唯一的，Kruskal 或 Prim 的选择不会出现同权替代。因此最小生成树唯一。

三、填空题 (20 分, 每题 2 分)

26. 若经常需要在线性表头部进行插入和删除操作, 最合适的存储结构是 _____; 若需要随机存取, 最合适的存储结构是 _____。

解答:

答案: 链式存储; 顺序存储。在线性表头部频繁插入、删除时, 链表只需修改引用, 不必整体移动大量元素; 随机存取则要求能按下标直接定位, 顺序表可在 $O(1)$ 时间访问任意位置。

27. 顺序表中 a, b, c, d, e, f 的查找概率分别为 0.12, 0.25, 0.23, 0.20, 0.05, 0.15。为了使查找成功的平均比较次数最少, 数据元素的存放顺序应为 _____。

解答:

答案: b, c, d, f, a, e 。顺序查找成功的平均比较次数为 $\sum p_i c_i$, 越靠前的元素比较次数越少。因此应按查找概率从大到小排列: $b(0.25), c(0.23), d(0.20), f(0.15), a(0.12), e(0.05)$ 。

28. 含 120 个结点的完全二叉树共有 _____ 层, 有 _____ 个叶子结点。

解答:

答案: 7, 60。按根为第 1 层计, $2^6 - 1 = 63 < 120 \leq 127 = 2^7 - 1$, 故共有 7 层。完全二叉树中编号大于 $\lfloor n/2 \rfloor$ 的结点为叶子, 叶子数为 $120 - \lfloor 120/2 \rfloor = 60$ 。

29. 已知一棵树的中序遍历序列为 DBGEACF、后序遍历序列为 DGEBFCA, 其前序遍历结果为 _____。

解答:

答案: ABDEGCF。后序最后一个结点 A 为根; 中序按 A 分为左子树 DBGE 和右子树 CF。左子树后序 DGEB 的根为 B, 再递归得到 B, D, E, G; 右子树根为 C, 右孩子为 F。合并得到前序 ABDEGCF。

30. 字符 a, b, c, d 的出现次数分别为 1000, 2000, 6000, 1000, 采用 Huffman 编码时, 该电文总长度为 _____ 比特。

解答:

答案: 16000。Huffman 合并过程为: 1000 与 1000 合并成 2000, 两个 2000 合并成 4000, 再与 6000 合并成根。于是频率为 6000 的字符码长为 1, 频率为 2000 的字符码长为 2, 两个频率为 1000 的字符码长为 3。总长度为 $6000 \times 1 + 2000 \times 2 + 1000 \times 3 + 1000 \times 3 = 16000$ 比特。

31. 对关键字序列 45, 80, 55, 40, 42, 85 从最后一个非叶子结点开始调整建最大堆, 得到的初始最大堆为 _____。

解答:

答案: 85, 80, 55, 40, 42, 45。按顺序存储视为完全二叉树, 从最后一个非叶子结点 55 开始下滤, 先与孩子 85 交换, 得到 45, 80, 85, 40, 42, 55; 再调整根 45, 与较大孩子 85 交换后继续下滤, 与 55 交换, 最终为 85, 80, 55, 40, 42, 45。

32. 一棵 m 叉树中度数为 i 的结点数为 N_i , 则终端结点数为 _____。

解答:

答案: $1 + \sum_{i=2}^m (i-1)N_i$ 。设终端结点数为 N_0 。树的边数等于结点总数减 1, 也等于所有结点度数之和: $\sum_{i=1}^m iN_i = \sum_{i=0}^m N_i - 1$ 。整理并消去 N_1 , 得 $N_0 = 1 + \sum_{i=2}^m (i-1)N_i$ 。

33. 散列表长 $m = 15$ ，散列函数 $H(\text{key}) = \text{key} \% 11$ ，已有地址 $\text{addr}(16) = 5$ 、 $\text{addr}(37) = 4$ 、 $\text{addr}(50) = 6$ 、 $\text{addr}(70) = 7$ 。若用线性探查处理冲突，关键字 38 的地址为 _____。

解答：

答案：8。先算 $H(38) = 38 \% 11 = 5$ 。地址 5 已被 16 占用，线性探查依次检查 6、7，也已分别被 50、70 占用；继续探查地址 8，为空，因此 38 存放在地址 8。

34. 森林 F 有 4 棵树，结点数分别为 10, 9, 11, 7。转成一棵二叉树后，其根结点右子树中有 _____ 个结点。

解答：

答案：27。森林转二叉树时，第一棵树的根作为二叉树根，其余树的根沿右孩子链连接。因此二叉树根结点的右子树包含第 2、第 3、第 4 棵树的全部结点，共 $9 + 11 + 7 = 27$ 个。

35. 对无向图，若边数 $m \geq n$ (n 为结点数)，则该图一定 _____。

解答：

答案：存在回路。无环无向图是森林。含 n 个顶点、 c 个连通分量的森林最多有 $n - c \leq n - 1$ 条边。若边数 $m \geq n$ ，超过了无环图的最大可能边数，因此图中必定存在回路。

四、简答题 (3 题，共 14 分)

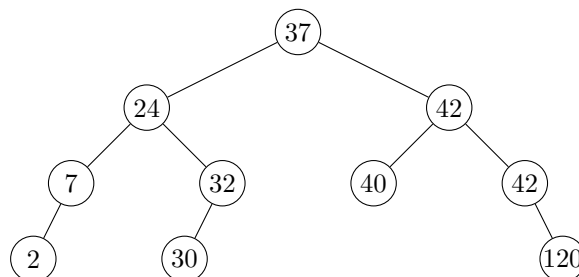
36. 对序列 $(H, E, B, L, G, A, F, J, I, C, D, K)$ 按字母升序排序，求：

- (1) 冒泡排序第一趟交换结束后的结果；
- (2) 二路归并排序第一趟归并后的结果；
- (3) 初始步长为 4 的 Shell 排序第一趟后的结果。

解答：

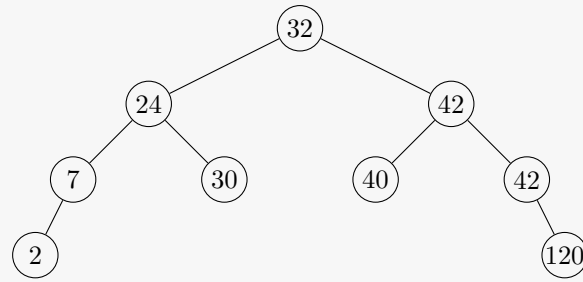
- (1) 冒泡排序第一趟：(E, B, H, G, A, F, J, I, C, D, K, L)。
- (2) 二路归并第一趟：(E, H, B, L, A, G, F, J, C, I, D, K)。
- (3) Shell 排序步长为 4 的第一趟：(G, A, B, J, H, C, D, K, I, E, F, L)。

37. 在题图给出的二叉排序树中删除结点 37。要求先寻找该结点左子树上的最大者 r ，并利用 r 辅助删除该结点。请画出删除后的二叉排序树结构。



解答：

左子树最大结点为 32。用 32 替代根结点 37，并删除原来 32 所在位置；其左孩子 30 接到结点 24 的右孩子位置。删除后的二叉排序树如下：



38. Dr. Stranger 的电脑感染了一种病毒。该病毒会把文档中的每种字母固定替换为另一种字母，且互不相同，可以看作字母集合上的双射函数。已知字母集合 $X = \{a, b, c, d, e\}$ ，文档感染后内容为

$$D' = \{\text{cedbdac}, \text{cac}, \text{ecd}, \text{dca}, \text{aba}, \text{bac}\}.$$

为使题目可唯一作答，补充条件：病毒替换规则是字母表 $a \rightarrow b \rightarrow c \rightarrow d \rightarrow e \rightarrow a$ 的循环置换，即

$$f(a) = b, \quad f(b) = c, \quad f(c) = d, \quad f(d) = e, \quad f(e) = a.$$

求解密函数 f^{-1} ，并还原原文档 D 。

解答：

解密函数为

$$f^{-1}(a) = e, \quad f^{-1}(b) = a, \quad f^{-1}(c) = b, \quad f^{-1}(d) = c, \quad f^{-1}(e) = d.$$

逐字解密得到

$$D = \{\text{bdcacbe}, \text{beb}, \text{dbc}, \text{cbe}, \text{eae}, \text{aeb}\}.$$

若没有上述补充条件，仅凭密文集合 D' 无法唯一确定替换函数。

五、算法填空 (4 题，共 26 分)

39. (只带尾引用的循环单链表) 程序描述只带尾引用的循环单链表操作：先把 n 个整数依次插入链表头部，然后删除链表尾部 m 个元素，再在链表前部插入 k 个整数。补全程序空白。

```

class Node:
    def __init__(self, data):
        self.data = data
        self.next = None

class LinkedList:
    def __init__(self):
        self.tail = None          # 只保存尾结点引用

    def push_front(self, data): # 在链表头部插入元素
        nd = Node(data)
        if self.tail is None:
            self.tail = nd
            nd.next = nd
        else:
            nd.next = _____ # (1)
            self.tail.next = nd

    def pop_back(self):         # 在链表尾部删除元素
        if self.tail is None:
            return
        if _____:         # (2)

```

```

        self.tail = None
        return

    ptr = self.tail.next
    while ____:                # (3)
        ptr = ptr.next
    ptr.next = self.tail.next
    ____                        # (4)

def to_list(self):
    if self.tail is None:
        return []
    ans = []
    ptr = self.tail.next
    while True:
        ans.append(ptr.data)
        ptr = ptr.next
        if ____:                # (5)
            break
    return ans

```

解答：

空白处依次为：(1) `self.tail.next`; (2) `self.tail.next is self.tail`; (3) `ptr.next is not self.tail`; (4) `self.tail = ptr`; (5) `ptr is self.tail.next`。

只保存尾引用时，头结点就是 `self.tail.next`。头插时新结点先指向原头结点，再由尾结点指向新结点。尾删时需要从头结点出发找到尾结点的前驱 `ptr`，再令 `ptr.next` 指回头结点，并把 `tail` 更新为 `ptr`。输出时从头结点开始，走回头结点即停止。

40. (二叉树宽度) 给定一棵二叉树，宽度定义为结点最多的那一层的结点数。输入每个结点的左右儿子编号，若为 -1 表示没有相应儿子，补全程序求二叉树宽度。

```

class BinaryTree:
    def __init__(self, data=None):
        self.data = data
        self.left = None
        self.right = None
        self.parent = -1

def traversal(root, level):
    if root is None:
        return
    width[level] += 1
    ____                # (1)
    ____                # (2)

n = int(input())
nodes = [BinaryTree(i) for i in range(n)]
width = [0] * max(1, n)
for nd in range(n):
    L, R = map(int, input().split())
    nodes[nd].left = None if L == -1 else nodes[L]
    nodes[nd].right = None if R == -1 else nodes[R]
    if L != -1:
        ____            # (3)
    if R != -1:

```

```

            ----- # (4)

root = None
for i in range(n):
    if -----: # (5)
        root = nodes[i]
        break

traversal(root, 0)
print(max(width))

```

解答:

空白处依次为:(1) traversal(root.left, level + 1);(2) traversal(root.right, level + 1);
(3) nodes[L].parent = nd; (4) nodes[R].parent = nd; (5) nodes[i].parent == -1。
递归遍历时, 用 level 表示当前层号, 每访问一个结点就把对应层的计数加 1。建立二叉树时记录每个结点的父结点编号, 最后父结点仍为 -1 的结点就是根结点。

41. (Prim 算法) 用 Prim 算法求无向图最小生成树。图用邻接矩阵 G 存储, $G[u][v] = \text{None}$ 表示无边, 顶点编号从 0 开始。补全程序, 使其返回最小生成树的边列表。

```

def prim(G):
    INF = float("inf")
    n = len(G)
    dist = [INF] * n # 各顶点到已建树部分的最短距离
    used = [False] * n # 顶点是否已加入最小生成树
    prev = [None] * n # 顶点 v 通过边 (v, prev[v]) 加入树
    edges = []
    done_num = 0

    while -----: # (1)
        if done_num == 0:
            x, min_dist = 0, 0
        else:
            x, min_dist = None, INF
            for i in range(n):
                if (not used[i]) and -----: # (2)
                    x, min_dist = i, dist[i]

            ----- # (3)
        done_num += 1
        if done_num > 1:
            edges.append(-----) # (4)

        for v in range(n):
            if (not used[v]) and G[x][v] is not None and -----: # (5)
                dist[v] = G[x][v]
                ----- # (6)

    return edges

```

解答:

空白处依次为:(1) done_num < n;(2) dist[i] < min_dist;(3) used[x] = True;(4) (x, prev[x]);
(5) G[x][v] < dist[v]; (6) prev[v] = x。
第一个加入生成树的顶点取 0。之后每轮从未加入的顶点中选择 dist 最小者 x 加入; 若 x 不是第一个顶点, 则边 (x, prev[x]) 是新加入的最小生成树边。随后用 x 更新所有未加入顶点的最短连接

边。

42. (连通分量大小) 补全程序，计算一个无向图中所有连通分量的结点数。输入为图的结点数和边列表，输出每个连通分量的结点数。

```
def dfs(node, visited, graph, n):
    count = 1
    visited[node] = True
    for i in range(n):
        if i in graph[node] and ____:      # (1)
            count += ____                  # (2)
    return count

def count_components(n, edges):
    graph = {i: [] for i in range(n)}
    for u, v in edges:
        ____                              # (3)
        ____                              # (4)

    visited = [False] * n
    components = []
    for i in range(n):
        if ____:                          # (5)
            components.append(____)       # (6)
    return components

n = int(input())
edges = eval(input())
print(count_components(n, edges))
```

解答：

空白处依次为：(1) not visited[i]; (2) dfs(i, visited, graph, n); (3) graph[u].append(v); (4) graph[v].append(u); (5) not visited[i]; (6) dfs(i, visited, graph, n)。
无向边需要在邻接表中双向加入。每次从尚未访问的顶点开始 DFS，返回本次 DFS 访问到的顶点数，这个数就是一个连通分量的大小。

数据结构与算法 B

common-模拟题 1-期末

一、选择题 (每小题 3 分, 共 30 分)

1. 数据结构的三个基本要素是 ()。

- | | |
|--------------------|--------------------|
| (A) 顺序结构、链式结构、散列结构 | (C) 线性结构、树型结构、图状结构 |
| (B) 逻辑结构、存储结构、算法 | (D) 以上都不是 |

解答:

选 B。数据结构三要素为逻辑结构、存储结构和算法 (数据操作)。

2. 栈是一种后进先出表。若入栈的顺序为 {1, 2, 3, 4}, 则出栈的顺序不可能是 ()。

- | | |
|----------|----------|
| (A) 1234 | (C) 1423 |
| (B) 1243 | (D) 2134 |

解答:

选 C。1423 不可能: 要 4 在 2 之前出栈, 则 4 入栈时 2, 3 已在栈中, 出栈顺序应为 4, 3, 2 而非 4, 2, 3。

3. 用一个大小为 6 的数组来实现环形队列, 元素从下标 0 开始存放。当前 front 和 rear 的值分别为 4 和 1, 现在从队列中删除一个元素, 再增加两个元素, 则 front 和 rear 的值为 ()。

- (A) 0, 2
(B) 5, 3
(C) 0, 1
(D) 5, 2

解答:

选 B。环形队列删除: $\text{front} = (4+1)\%6 = 5$ 。增加两个: $\text{rear} = (1+2)\%6 = 3$ 。

4. 假定一棵二叉树的结点个数为 33 个, 定义根结点的层数为 0, 则它的高度最小为 ()。

- | | |
|-------|-------|
| (A) 5 | (C) 7 |
| (B) 6 | (D) 4 |

解答:

选 A。根结点层数为 0 时, 含 33 个结点的二叉树最小最大层数为 5, 因为前 5 层最多有 $2^5 - 1 = 31$ 个结点, 而前 6 层最多有 $2^6 - 1 = 63$ 个结点。

5. 已知某二叉树的先根周游序列为 {A, B, D, E, C}, 中根周游序列为 {D, B, E, A, C}, 则该二叉树的后根周游序列是 ()。

- | | |
|---------------------|---------------------|
| (A) {A, B, C, D, E} | (C) {C, D, E, B, A} |
| (B) {D, B, E, C, A} | (D) {D, E, B, C, A} |

解答:

选 D。由先序确定根 A, 中序划分左子树 {D, B, E}、右子树 {C}。递归可得后序: D, E, B, C, A。

6. 在待排序的元素序列基本有序的前提下, 下列空间效率最差的排序方法是 ()。

- (A) 直接插入排序 (C) 快速排序
(B) 选择排序 (D) 冒泡排序

解答：

选 C。快速排序在基本有序时递归深度 $O(n)$ ，空间复杂度 $O(n)$ ，远高于其他 $O(1)$ 。

7. 在平均情况下，下列时间复杂度最好的排序方法是 ()。

- (A) 直接插入排序 (平均 $O(n^2)$) (C) 快速排序 (平均 $O(n \log n)$)
(B) 选择排序 (平均 $O(n^2)$) (D) 冒泡排序 (平均 $O(n^2)$)

解答：

选 C。快速排序平均时间复杂度 $O(n \log n)$ 。

8. 下列关于图的说法不正确的是 ()。

- (A) 用邻接矩阵法存储一个图时，其存储空间与图的边数无关 (C) 强连通分量是有向图中的极大强连通子图
(B) AOV 网络拓扑排序的结果是唯一的 (D) 一个有向图的邻接表和逆邻接表中的结点个数一定相等

解答：

选 B。拓扑排序结果不一定唯一，取决于入度为 0 的顶点选择顺序。

9. AVL 树是一种 ()。

- (A) 生成树 (C) B 树
(B) 平衡树 (D) 二叉排序树

解答：

选 B。AVL 树是平衡二叉搜索树 (任一结点左右子树高度差 ≤ 1)。

10. 任何一个无向连通图的最小生成树 ()。

- (A) 只有一棵 (C) 一定有多棵
(B) 有一棵或多棵 (D) 可能不存在

解答：

选 B。当图中有权值相等的边时，最小生成树可能不唯一。

二、解答题 (共 40 分)

11. (线性表存储结构比较) (8 分)

请说明线性表的顺序存储 (顺序表) 与链式存储 (单链表) 的异同, 比较它们的优缺点。当字典采用二分法进行检索时, 应采用哪一种存储结构?

解答:

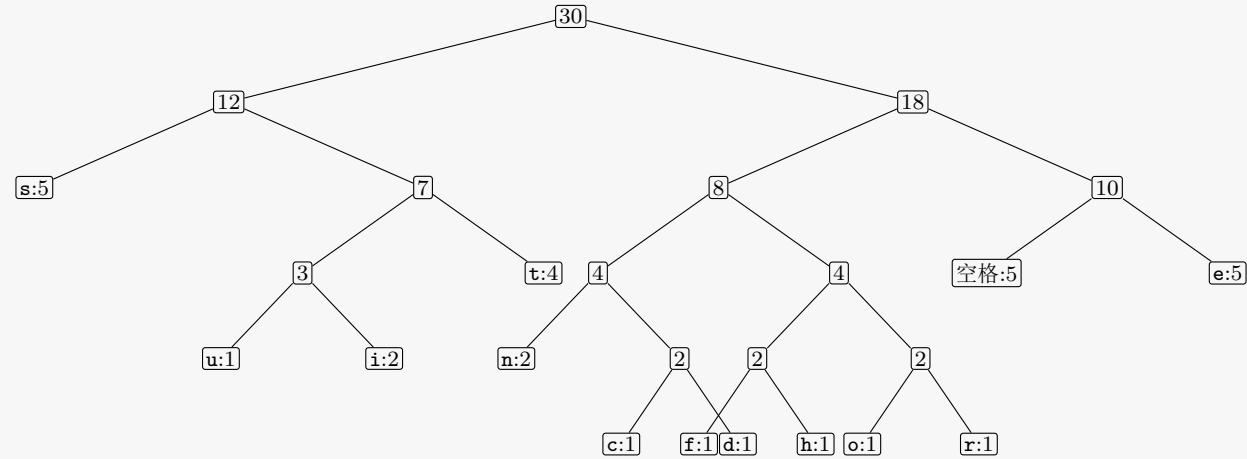
- 顺序表: 逻辑上相邻的元素在物理上也相邻, 支持随机访问 $O(1)$, 插入/删除需移动元素 $O(n)$ 。
- 单链表: 逻辑上相邻的元素物理上不一定相邻, 通过引用链接, 不支持随机访问 $O(n)$, 插入/删除只需修改引用 $O(1)$ 。
- 二分法检索需要随机访问中间元素, 应采用顺序表。

12. (Huffman 编码) (10 分)

字符串 "this sentence is used for test", 请使用 Huffman 编码方式对该字符串中的所有字符重新编码, 以实现对其的压缩, 给出编码过程中实现的 Huffman 树以及字符 's' 的编码结果。

解答:

先统计字符频率: 空格、e、s 各 5 次, t 为 4 次, i、n 各 2 次, c,d,f,h,o,r,u 各 1 次。Huffman 编码不唯一, 下面给出一种合法编码:



字符	空格	c	d	e	f	h	i	n	o	r	s	t	u
频率	5	1	1	5	1	1	2	2	1	1	5	4	1
编码	110	10010	10011	111	10100	10101	0101	1000	10110	10111	00	011	0100

因此在这棵 Huffman 树中, 字符 s 的编码可取 00。若合并等权结点的先后不同, 具体码字会变, 但频率高的字符码长较短这一结论不变。

13. (排序算法) (12 分)

对待排序的关键码集合 {E, C, A, M, S, R, D, F} 按升序排序, 请叙述直接插入排序、堆排序、快速排序、归并排序的基本思路, 以及它们对上述数组第一趟扫描后的结果 (快速排序使用第一个元素作为分界点)。

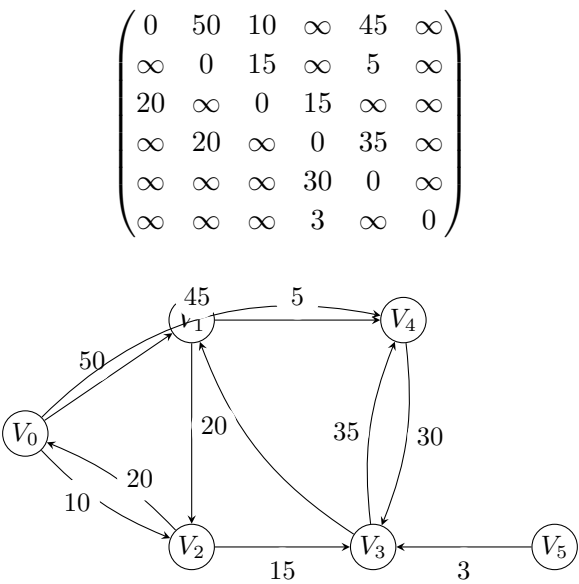
解答:

四种算法的第一趟结果如下表。这里“快速排序第一趟”按第一个元素 E 为枢轴、左右交替扫描的常见划分过程给出; “堆排序第一趟”常有两种口径, 表中同时列出。

算法	第一趟后的序列
直接插入排序	C, E, A, M, S, R, D, F
堆排序 (建成大根堆)	S, M, R, F, C, A, D, E
堆排序 (建堆后再输出一次最大值)	R, M, E, F, C, A, D, S
快速排序	D, C, A, E, S, R, M, F
归并排序	C, E, A, M, R, S, D, F

14. (最短路径) (10 分)

下面是一个带权有向图的邻接矩阵表示 (∞ 表示无边), 试画出该图结构, 并给出该图中由 V_0 到 V_4 的最短路径。



解答：
由矩阵可得有向边： $V_0 \rightarrow V_1(50)$ 、 $V_0 \rightarrow V_2(10)$ 、 $V_0 \rightarrow V_4(45)$ 、 $V_1 \rightarrow V_2(15)$ 、 $V_1 \rightarrow V_4(5)$ 、 $V_2 \rightarrow V_0(20)$ 、 $V_2 \rightarrow V_3(15)$ 、 $V_3 \rightarrow V_1(20)$ 、 $V_3 \rightarrow V_4(35)$ 、 $V_4 \rightarrow V_3(30)$ 、 $V_5 \rightarrow V_3(3)$ 。从 V_0 执行 Dijkstra 算法：

步骤	确定顶点	当前距离 (d_1, d_2, d_3, d_4, d_5)	前驱更新
初始	V_0	$(50, 10, \infty, 45, \infty)$	$pre_1 = 0, pre_2 = 0, pre_4 = 0$
1	V_2	$(50, 10, 25, 45, \infty)$	$pre_3 = 2$
2	V_3	$(45, 10, 25, 45, \infty)$	$pre_1 = 3$
3	V_1	$(45, 10, 25, 45, \infty)$	无更新
4	V_4	$(45, 10, 25, 45, \infty)$	完成

故最短路径为 $V_0 \rightarrow V_4$ ，长度为 45。顶点 V_5 从 V_0 不可达，不影响该最短路径。

三、算法填空 (共 10 分)

15. (散列表检索) (10 分)

完成下列 Python 散列表检索算法。设散列函数为 h ，采用线性探查法解决冲突；EMPTY 表示空位置。函数返回二元组 (found, position)，其中 found 表示是否检索成功，position 表示查找成功位置或首次空位位置。

```
EMPTY = None

def linear_search(table, key, h):
    m = len(table)
    d = _____ # (1) 初始散列地址
    for _ in range(m):
        entry = table[d]
        if entry is not EMPTY and _____: # (2) 检索成功
            _____ # (3) 记录位置
            return True, position
        if entry is EMPTY:
            return False, d
        _____ # (4) 线性探查下一地址
    _____ # (5) 散列表已探查一周
```

解答：

填空答案：

(1) $h(key) \% m$

(2) `entry.key == key`

(3) `position = d`

(4) $d = (d + 1) \% m$

(5) `return False, None`

线性探查从散列地址 $h(key) \% m$ 开始；若当前位置非空且关键字等于待查关键字，则查找成功。若遇到空位置，说明关键字不在表中且可插入该位置；否则地址加 1 后对表长取模继续探查。若探查一周仍未找到且未遇到空位，则散列表已满，返回查找失败。

四、算法设计 (共 20 分)

16. (求子树深度) (20 分)

已知一棵树采用孩子兄弟表示法。请用 Python 设计函数，求树中以值为 x 的结点为根的子树深度。限定：值为 x 的结点在树中最多有一个。注意栈或队列等辅助结构可以直接引用，不用自己定义。

```
class CSNode:
    def __init__(self, info):
        self.info = info          # 结点数据
        self.first_child = None   # 第一个孩子
        self.next_sibling = None  # 下一个兄弟

p_mytree = ...                  # 树根引用
```

解答：

采用深度优先搜索：先在整棵树中寻找值为 x 的结点；若找到，再从该结点出发计算子树深度。孩子兄弟表示中，`first_child` 引用第一个孩子，`next_sibling` 引用下一个兄弟。

参考实现如下 (结点不存在时返回 0)：

```
def subtree_height(root):
    if root is None:
        return 0
    best = 0
    child = root.first_child
    while child is not None:
        best = max(best, subtree_height(child))
        child = child.next_sibling
    return best + 1

def find_node(root, x):
    if root is None:
        return None
    if root.info == x:
        return root
    child = root.first_child
    while child is not None:
        ans = find_node(child, x)
        if ans is not None:
            return ans
        child = child.next_sibling
    return None

target = find_node(p_mytree, x)
answer = 0 if target is None else subtree_height(target)
```

`find_node` 会沿“第一个孩子-下一个兄弟”结构搜索整棵树；`subtree_height` 枚举当前结点的所有孩子，取孩子子树高度最大值后加 1，因此得到的就是目标子树深度。